

CS707 – Generative Models for NLP and Multimodal Reasoning

2026-27 Term 1

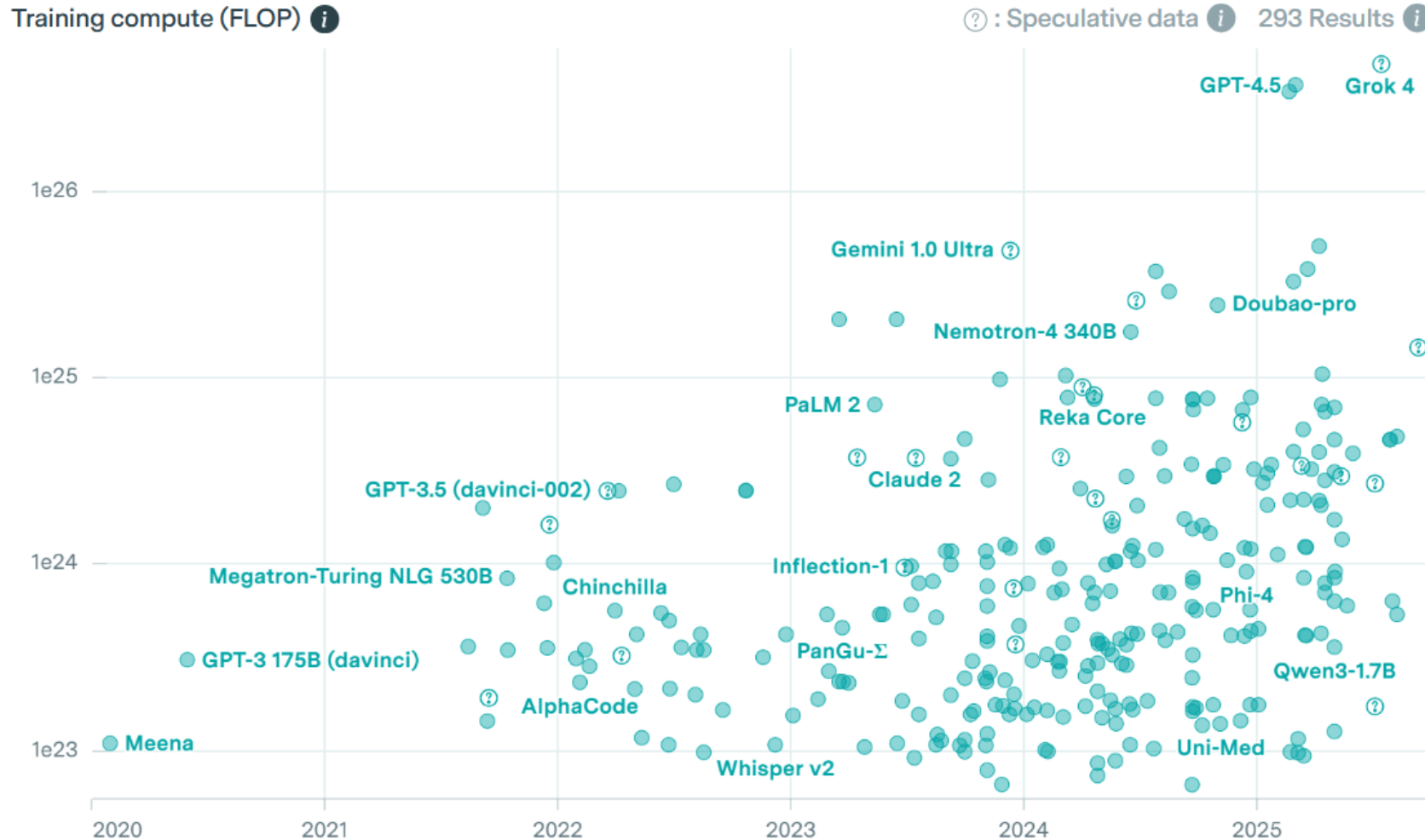
Week 4: Mixture of Experts (MoEs)

Dr. Wei Gao
Office: SCIS-1/4057
Email: weigao@smu.edu.sg
Tel.: 68261369

Announcement

- Quiz 1 next week (19 Sep)
- Bring your laptop
- Close book/notes
- Format: Paper reading comprehension (a paper will be handed out)
- Time: 45 minutes

Large Language Models



https://epoch.ai/data/ai-models?utm_source=chatgpt.com&view=graph&tab=large-scale

Training compute used for major LLMs has increased dramatically over time

Large Language Models

Model	Size	Approx. Training Time
RoBERTa-L	355M	410 A100s on 1 day
GPT-3	175B	2k A100s on 30 days
Switch Transformer	1.6T	2k A100s on 9 months!

VGG-Net
144M

RoBERTa-L
355M

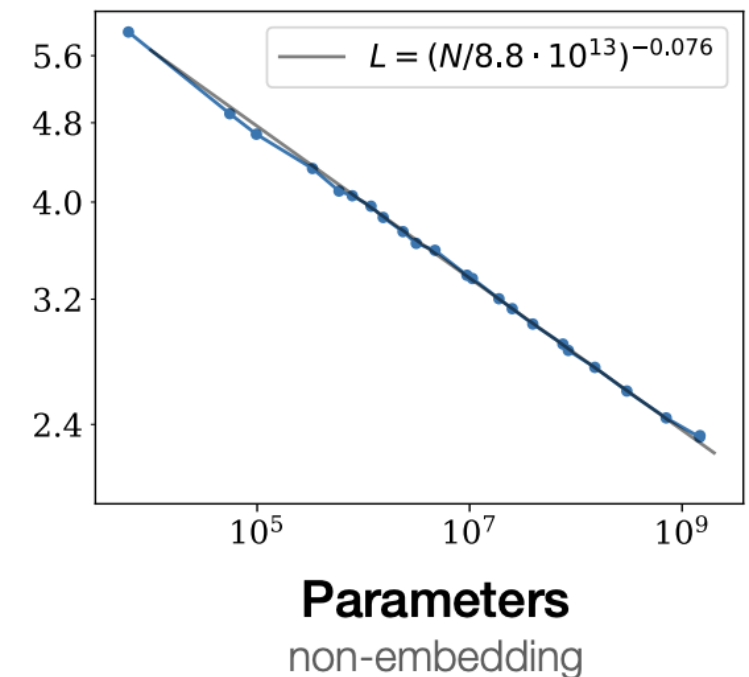
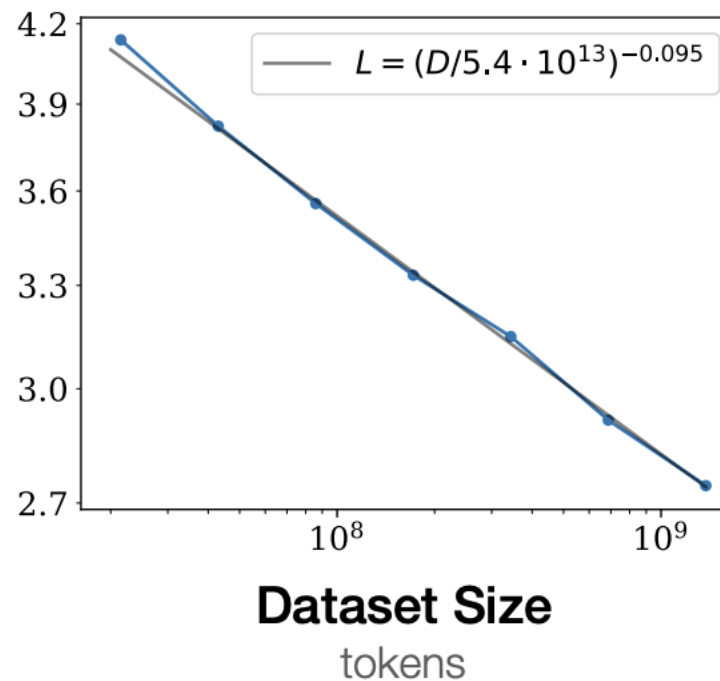
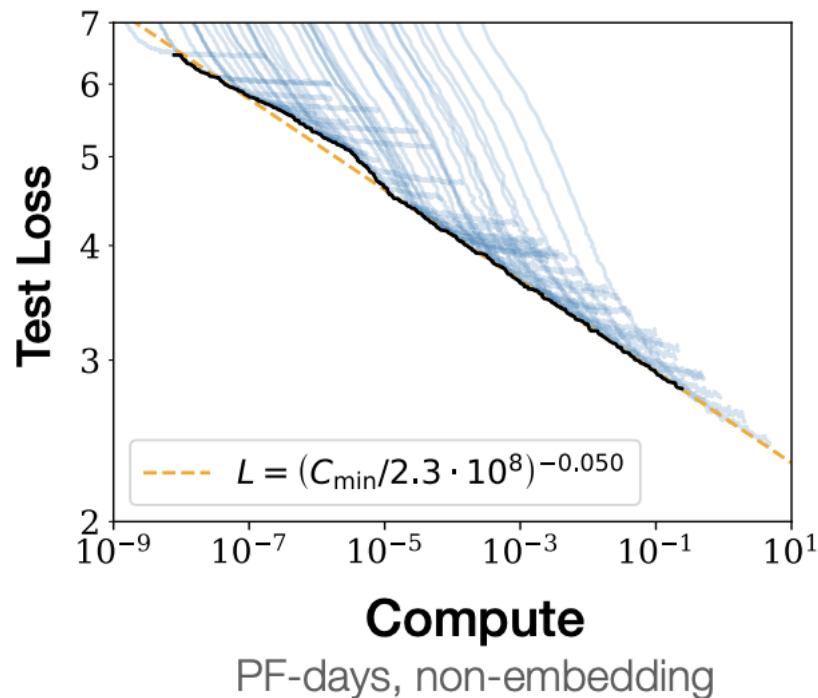
GPT2-XL
1.5B

GPT 3
175B

Switch Transformer
1.6 Trillion

Why care about model size?

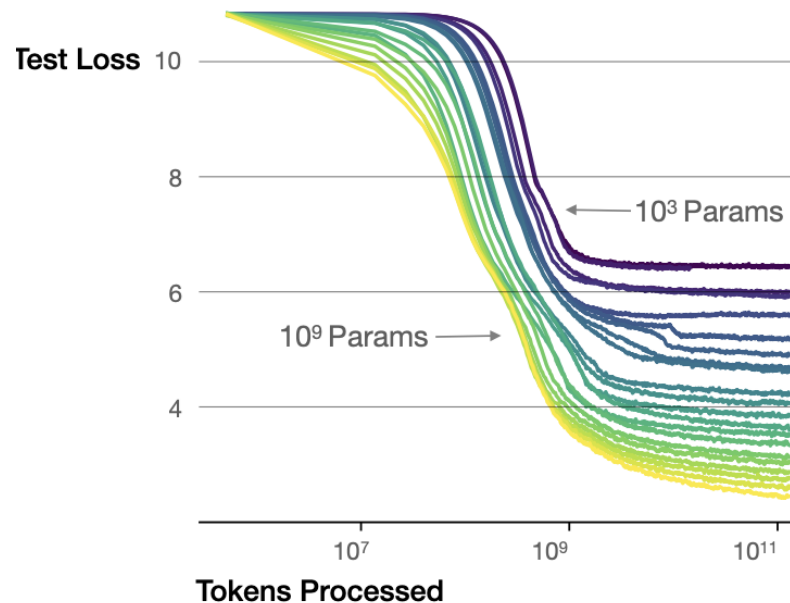
- Performance improve smoothly as we increase the model size, dataset size, and amount of compute used for training



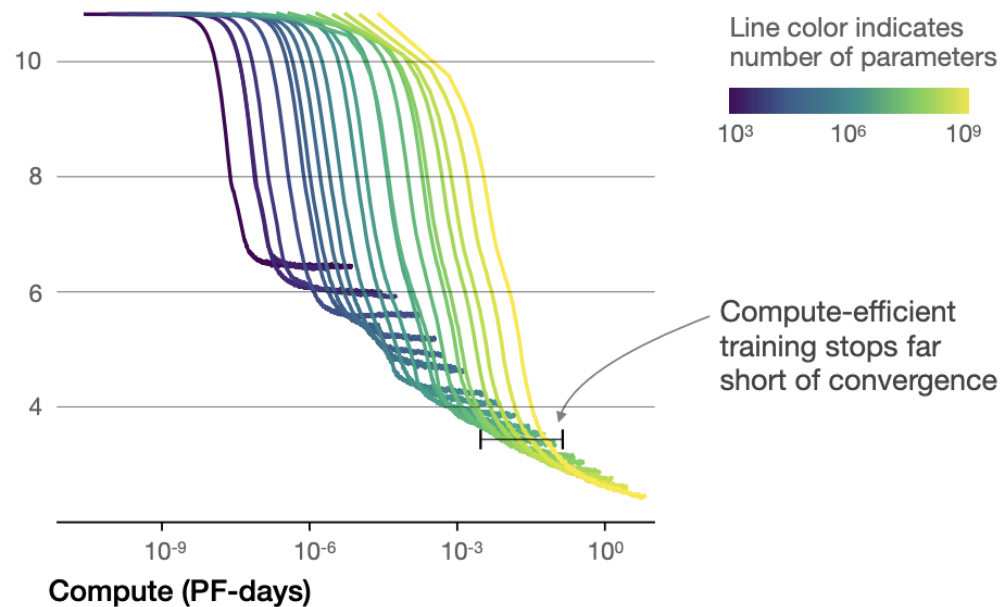
Neural Scale Laws

- Performance improve smoothly as we increase the model size, dataset size, and amount of compute used for training
- Large models are more sample efficient:** Given a fixed computing budget, training a larger model for fewer steps is better than training a smaller model for more steps.

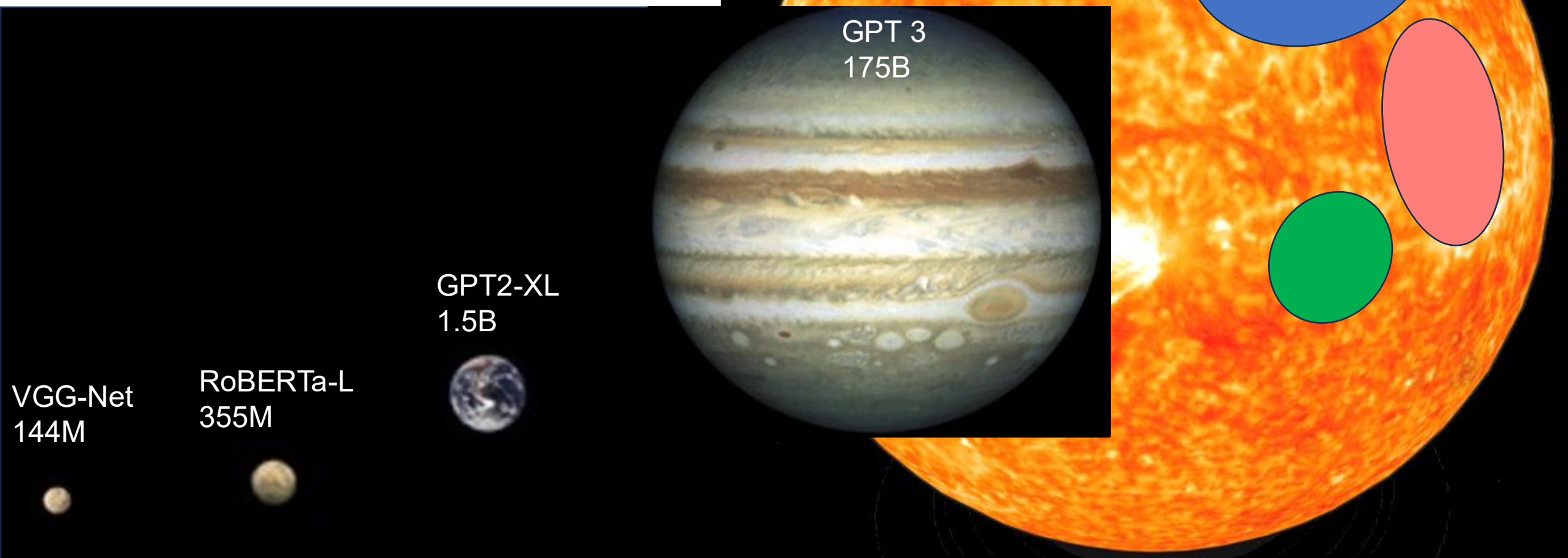
Larger models require **fewer samples** to reach the same performance



The optimal model size grows smoothly with the loss target and compute budget



How to efficiently increase model size?



Mixture of Experts

Adaptive Mixtures of Local Experts

Robert A. Jacobs

Michael I. Jordan

*Department of Brain and Cognitive Sciences, Massachusetts Institute of Technology,
Cambridge, MA 02139 USA*

Steven J. Nowlan

Geoffrey E. Hinton

*Department of Computer Science, University of Toronto,
Toronto, Canada M5S 1A4*

We present a new supervised learning procedure for systems composed of many separate networks, each of which learns to handle a subset of the complete set of training cases. The new procedure can be viewed either as a modular version of a multilayer supervised network, or as an associative version of competitive learning. It therefore provides a new link between these two apparently different approaches. We demonstrate that the learning procedure divides up a vowel discrimination task into appropriate subtasks, each of which can be solved by a very simple expert network.

Scaling Laws & Mixture of Experts (MoE)

Why MoE?

- Scaling laws → bigger models = lower loss, but compute cost grows sharply.
- Training dense trillion-parameter models is infeasible (compute cost bottleneck).

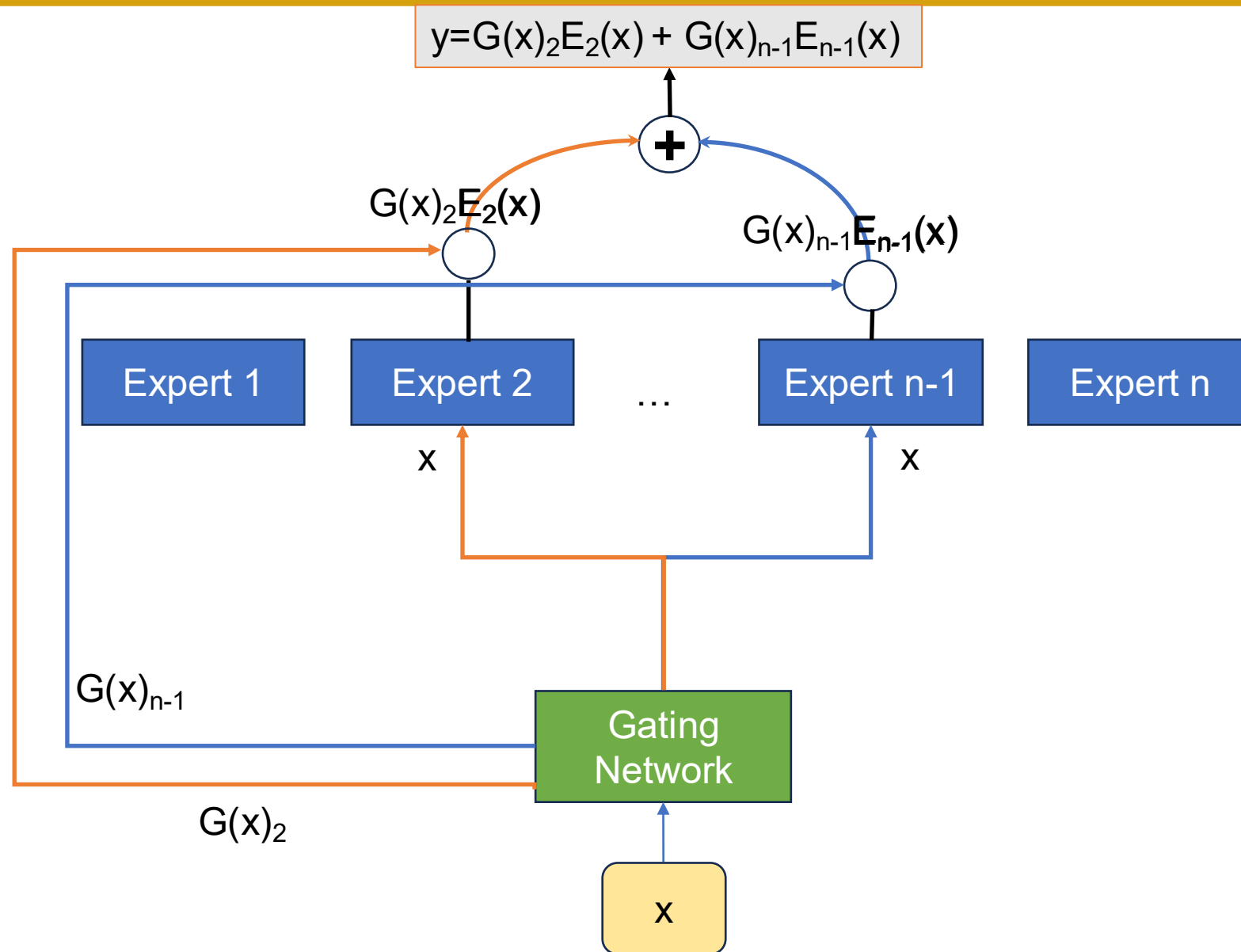
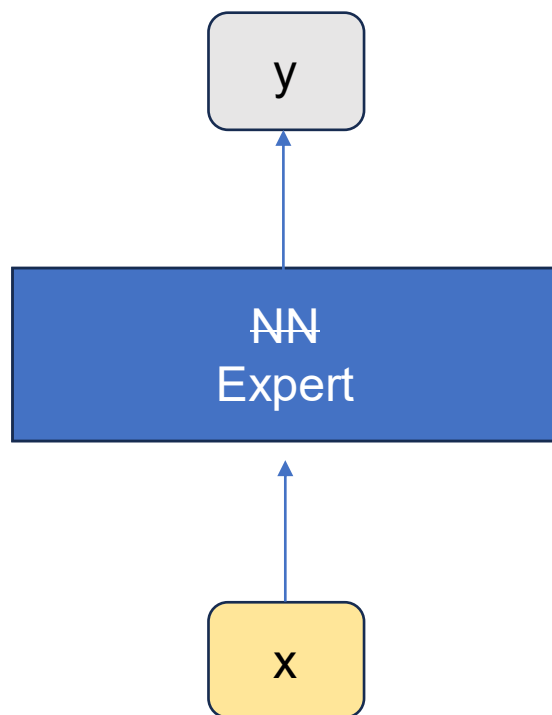
Key Idea

- Model has many "experts" (sub-networks).
- Only a **small subset** of experts activated per token, e.g.,
1T total params, but only ~10B active per forward pass.

Benefits

- **Decouples model size from compute cost.**
- Retains *capacity* of very large models.
- Achieves *efficiency*: more sample-efficient scaling without full dense compute.
- Enables trillion-parameter models within feasible budgets (Switch Transformer, GLaM).

Mixture of Experts

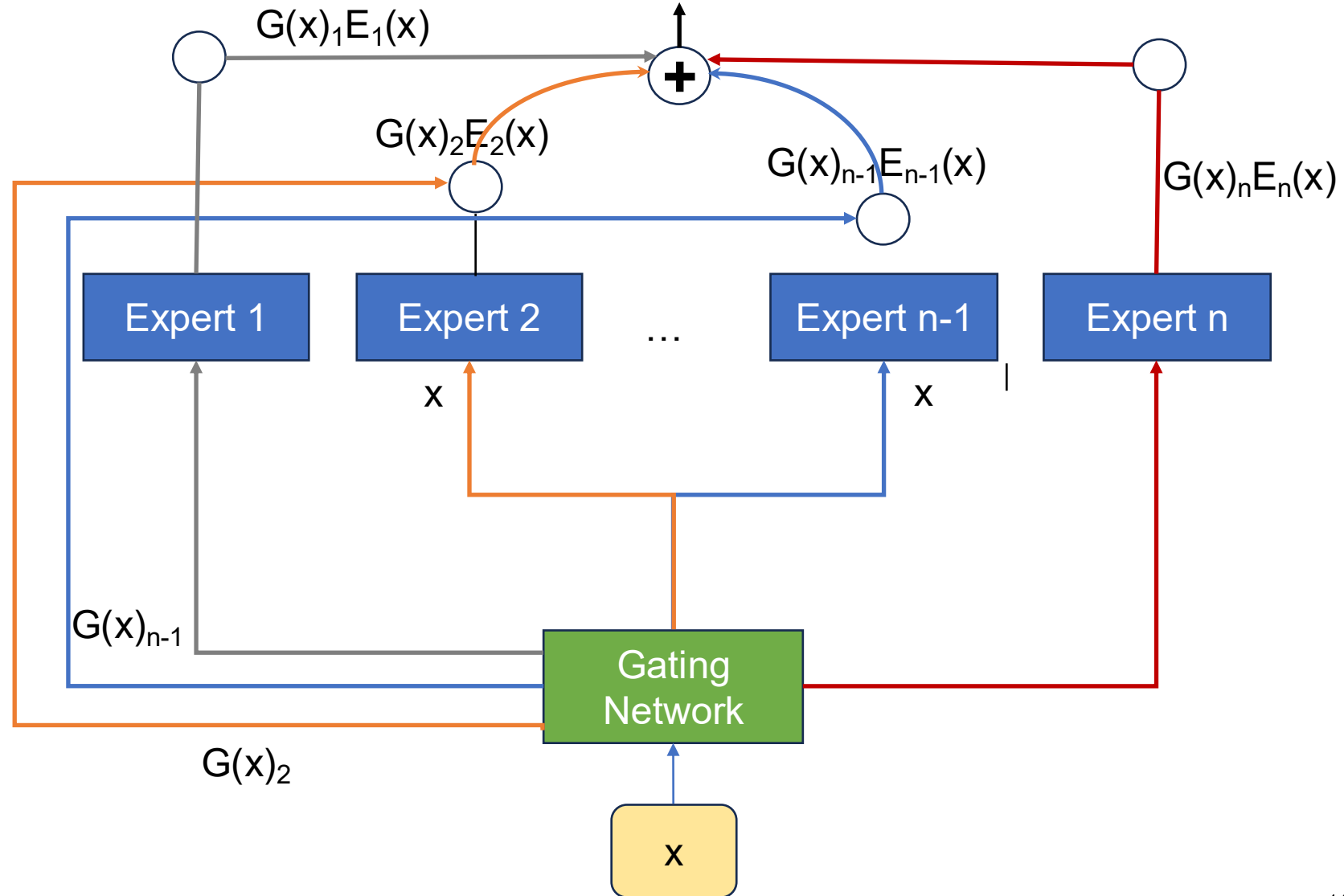


Mixture of Experts

$$y = G(x)_1 E_1(x) + G(x)_2 E_2(x) + \dots + G(x)_n E_n(x)$$

$$G(x) = \text{Softmax}(x \cdot W_g)$$

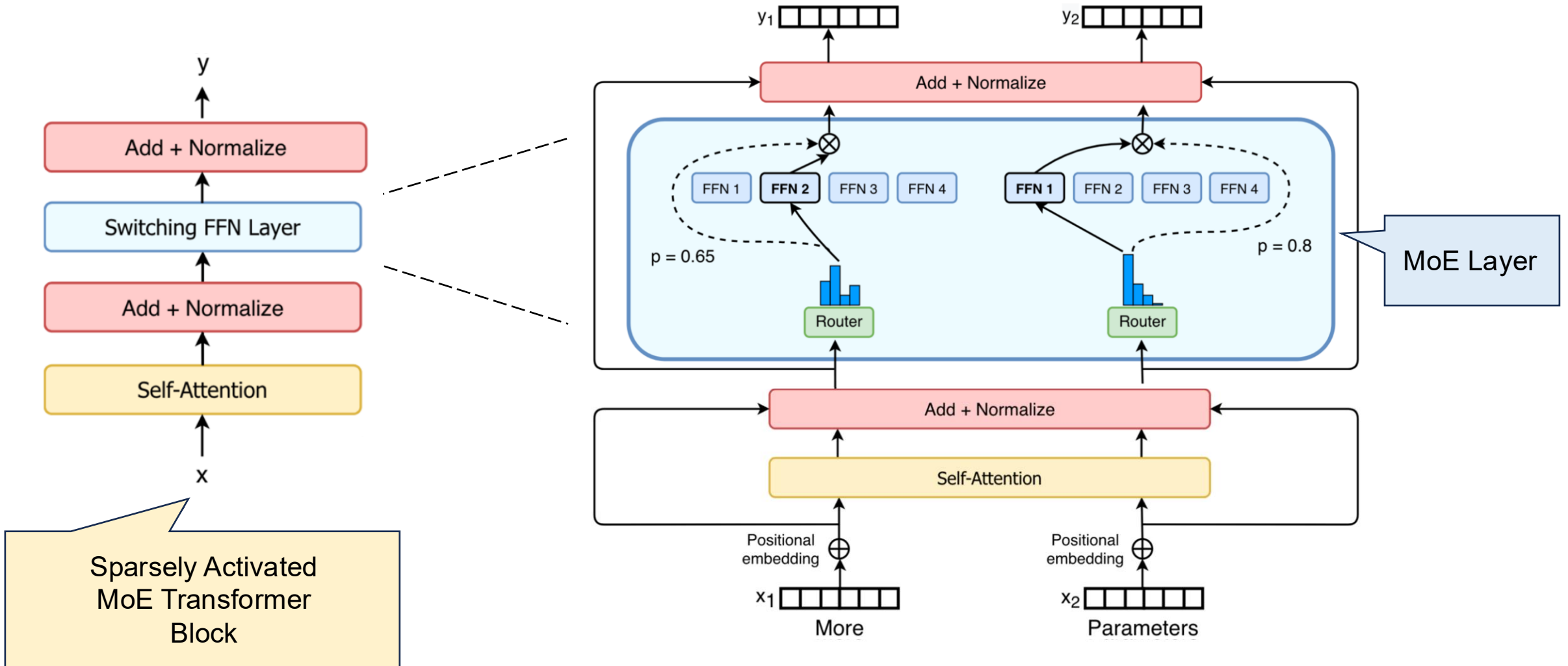
$$y = \sum_{i=1}^n G(x)_i E_i(x)$$



Mixture of Experts – Chronology

- Mixture of Experts Model (Jacobs et al., 1991; Jordan and Jacobs, 1994; Jordan et al., 1997; Tresp, 2001; Collobert et al., 2002)
- MoE layer in Deep Learning (Eigen et al., 2013; Shazeer et al., 2017; Dauphin et al., 2017; Vaswani et al., 2017)
- MoE layer in Transformer based LLMs (Fedus et al., 2021; Du, Nan, et al., 2021)
- Mixtral-8x7B (Jiang et al. 2024) - Apache 2.0 license; surpasses GPT-3.5 Turbo, Claude-2.1, Gemini Pro, and Llama 2 70B - chat model on human benchmarks

Mixture of Experts as a Layer

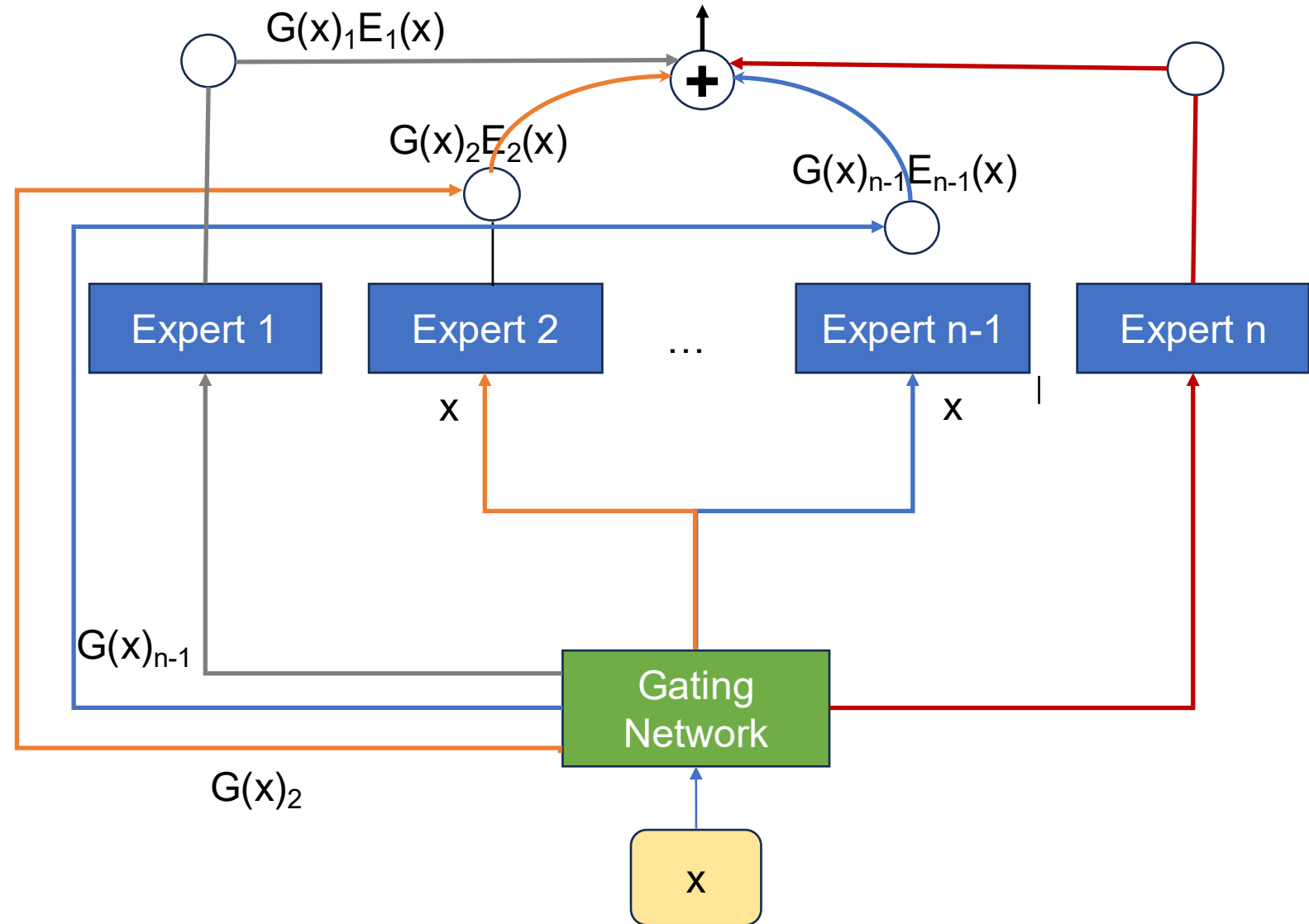


MoE Layer Backpropagation

$$y = G(x)_1 E_1(x) + G(x)_2 E_2(x) + \dots + G(x)_n E_n(x)$$

$$G(x) = \text{Softmax}(x \cdot W_g)$$

$$y = \sum_{i=1}^n \underbrace{G(x)_i E_i(x)}_{y_i}$$



MoE Layer Backpropagation

$$G(x) = \text{Softmax}(x \cdot W_g)$$

$$y = \sum_{i=1}^n G(x)_i E_i(x)$$

$$\frac{\Delta L}{\Delta E_i(x)} = G(x)_i \frac{\Delta L}{\Delta y}$$

d-dim

Scalar

d-dim vector

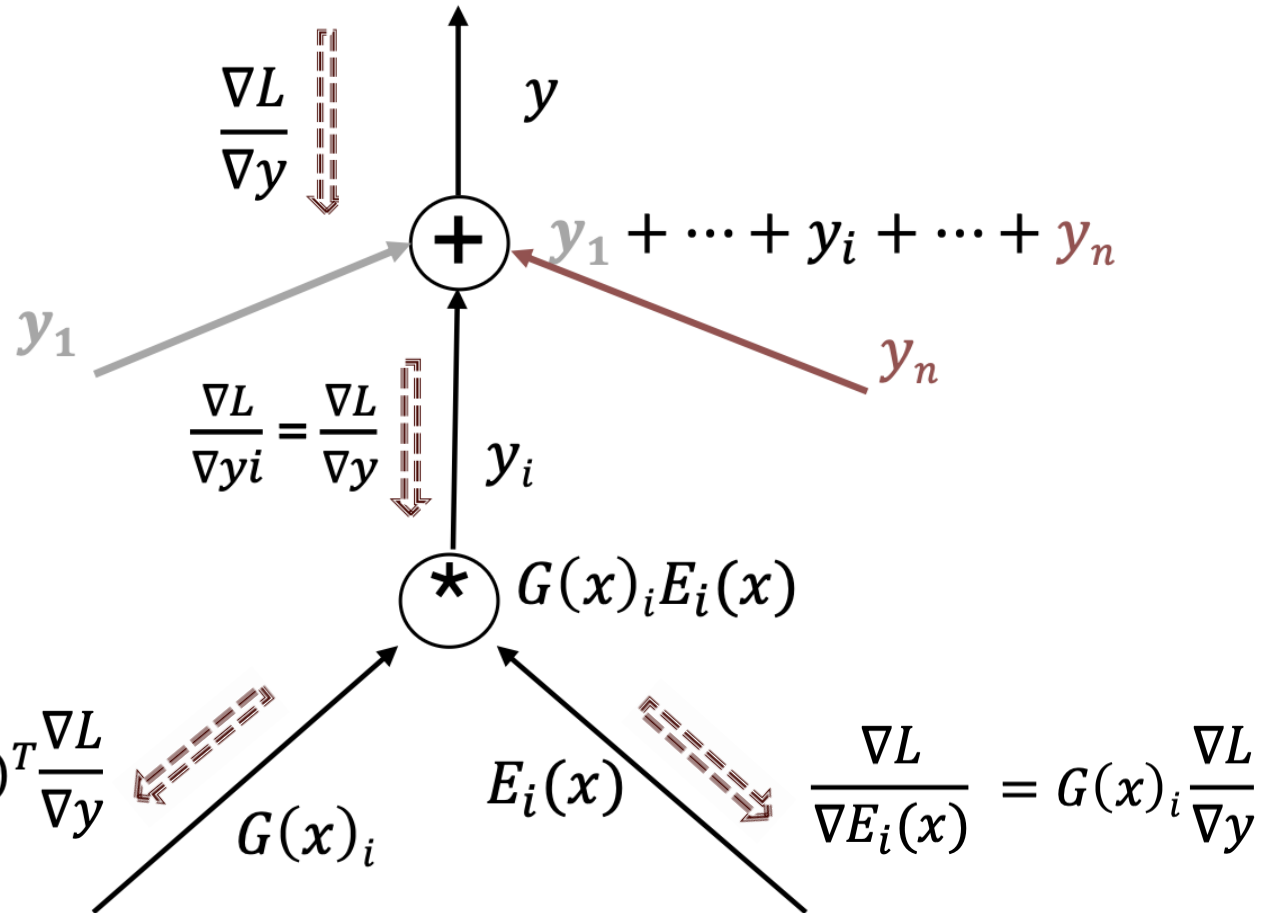
$$\frac{\Delta L}{\Delta G(x)_i} = (E_i(x))^T \frac{\Delta L}{\Delta y}$$

scalar

d-dim vector

d-dim vector

$$\frac{\nabla L}{\nabla G(x)_i} = (E_i(x))^T \frac{\nabla L}{\nabla y}$$



Sparse MoE Layer – Greedy expert selection

$$G(x) = \text{Softmax}(x \cdot W_g)$$

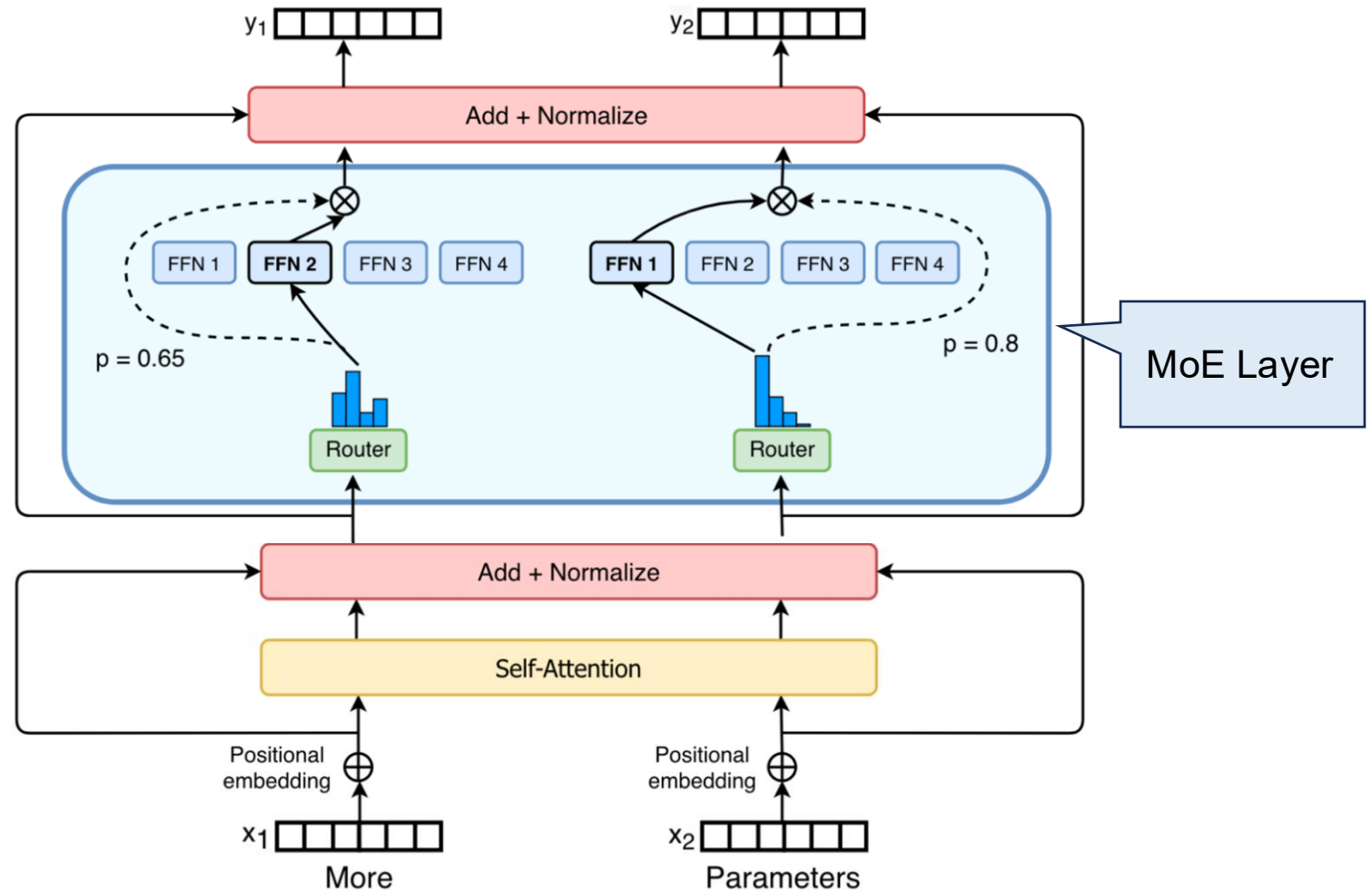
$$i^* = \text{argmax}_i G(x)_i$$

$$y = G(x)_{i^*} E_{i^*}(x)$$

$$\frac{\Delta L}{\Delta E_{i^*}(x)} = G(x)_{i^*} \frac{\Delta L}{\Delta y}$$

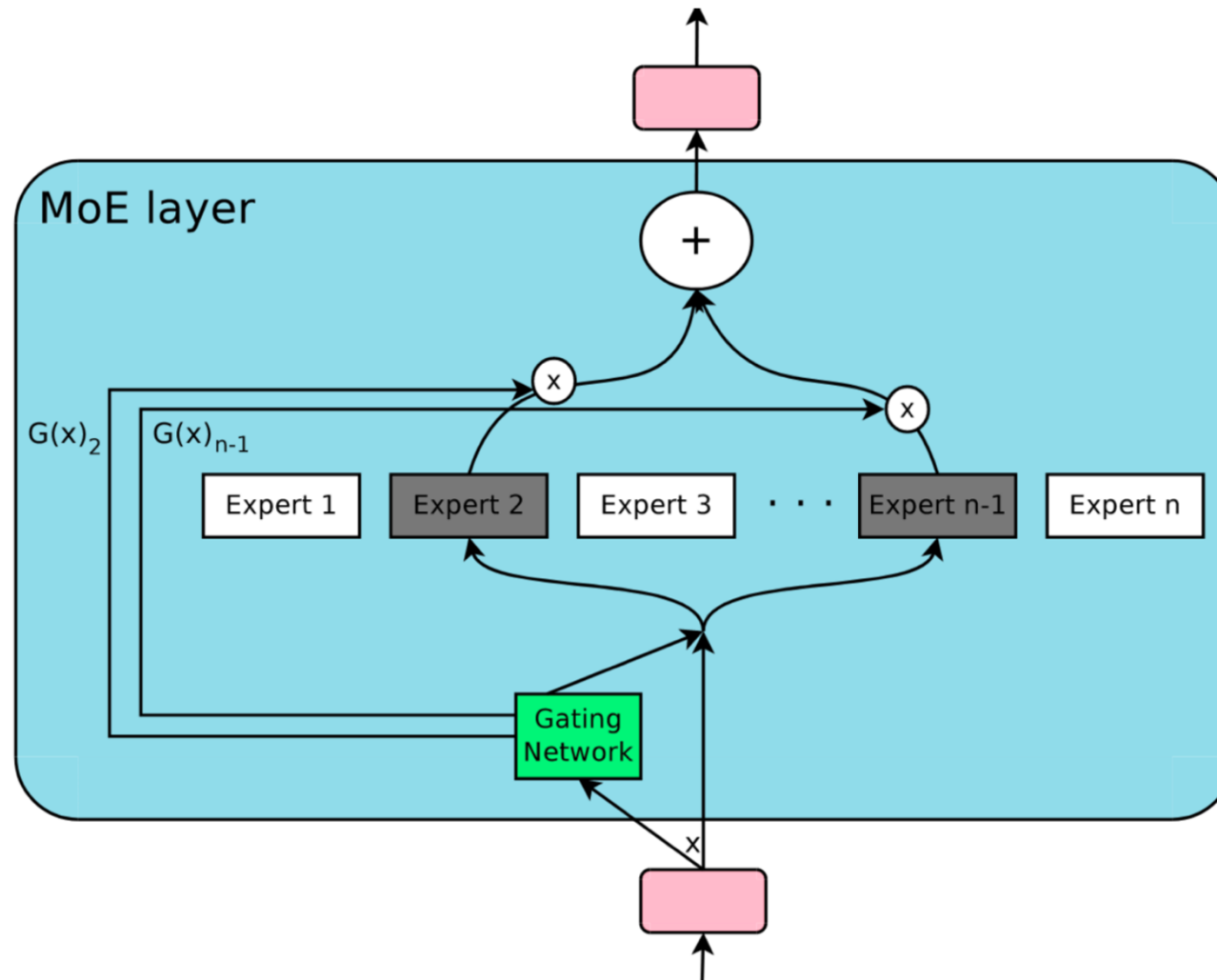
$$\frac{\Delta L}{\Delta G(x)_{i^*}} = (E_{i^*}(x))^T \frac{\Delta L}{\Delta y}$$

$$\frac{\Delta L}{\Delta E_i(x)} = \mathbf{0}; \frac{\Delta L}{\Delta G(x)_i} = 0 \text{ for } i \neq i^*$$



Only the selected expert gets trained on that input !

Sparse MoE Layer – Noisy Top-K Gating



Sparse MoE Layer – Noisy Top-K Gating

$$G(x) = \text{Softmax}(H(x))$$

$$H(x)_i = (x \cdot W_g)_i + \text{StandardNormal}() \cdot \text{Softplus}((x \cdot W_{noise})_i)$$

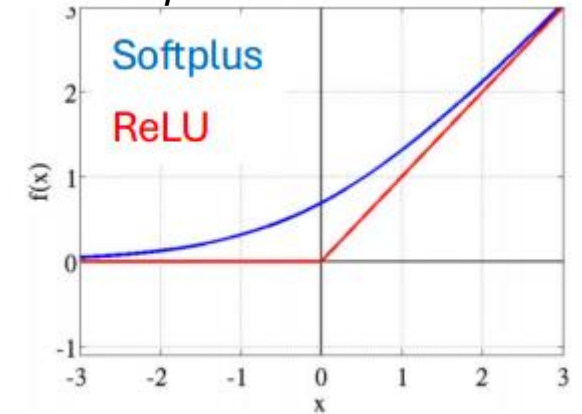
i-th logit of the Gating Network

Learnable parameter Mean

Learnable parameter Std.Dev

To avoid always picking the same expert deterministically, **noise is added to the logits**

$$\text{softplus}(x) = \frac{1}{\beta} * \log(1 + \exp(\beta * x))$$



$$\text{KeepTopK}(v, k)_i = \begin{cases} v_i, & \text{if } v_i \text{ is in the top } k \text{ elements of } v \\ -\infty, & \text{otherwise} \end{cases}$$

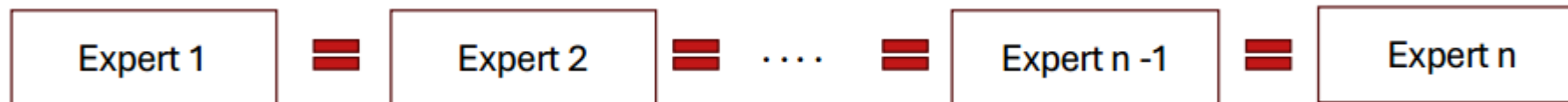
Ensure 0 probability after softmax

$$G(x) = \text{Softmax}(\text{KeepTopK}(H(x), k))$$

Only k non-zero elements; add up to 1

Why do experts not collapse in practice?

- All experts have:
 - Same architecture
 - Same initialization scheme
 - Trained with same optimizer
- So why don't they collapse? i.e.



Why do experts not collapse in practice?

Towards Understanding the Mixture-of-Experts Layer in Deep Learning

Zixiang Chen

Department of Computer Science
University of California, Los Angeles
Los Angeles, CA 90095, USA
chenzx19@cs.ucla.edu

Yihe Deng

Department of Computer Science
University of California, Los Angeles
Los Angeles, CA 90095, USA
yihedeng@cs.ucla.edu

Yue Wu

Department of Computer Science
University of California, Los Angeles
Los Angeles, CA 90095, USA
ywu@cs.ucla.edu

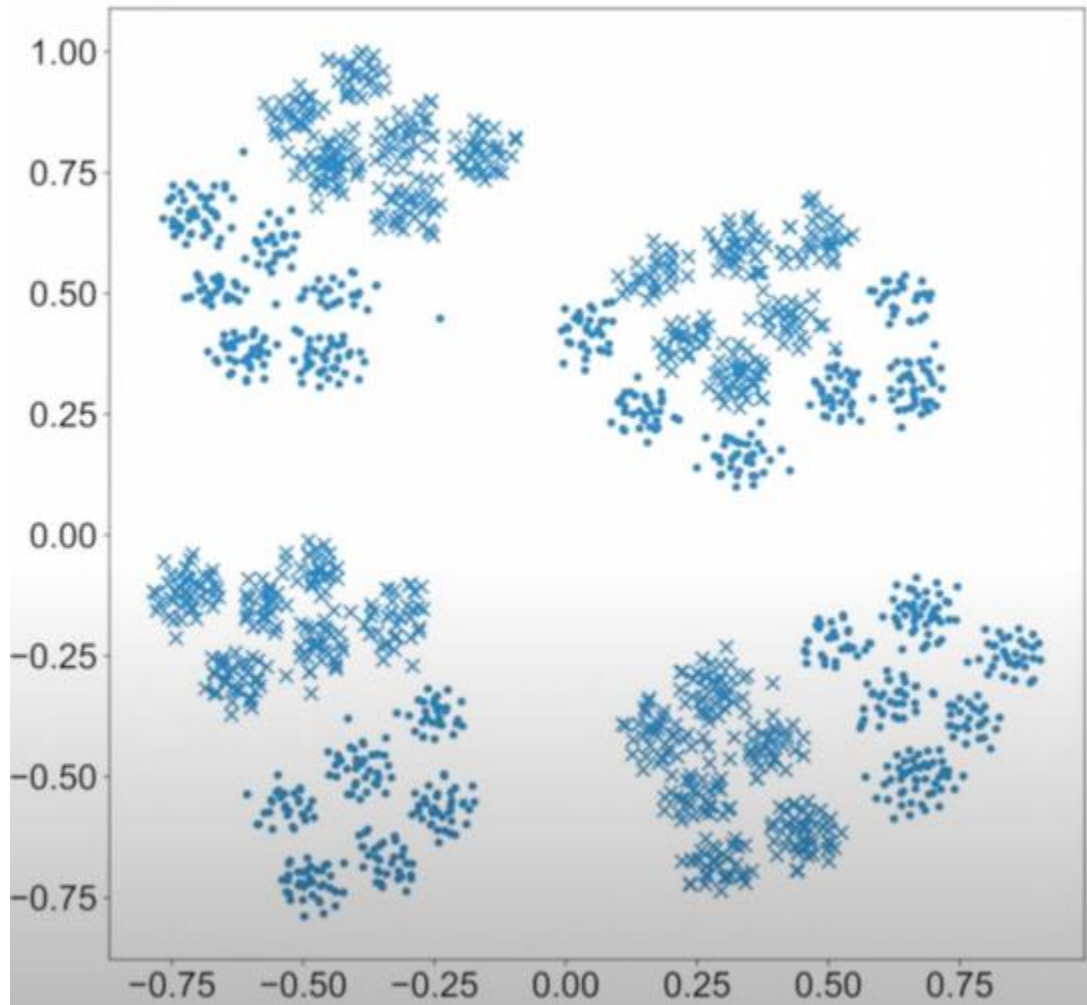
Quanquan Gu

Department of Computer Science
University of California, Los Angeles
Los Angeles, CA 90095, USA
qgu@cs.ucla.edu

Yuanzhi Li

Machine Learning Department
Carnegie Mellon University
Pittsburgh, PA 15213, USA
yuanzhil@andrew.cmu.edu

Why do experts not collapse in practice?



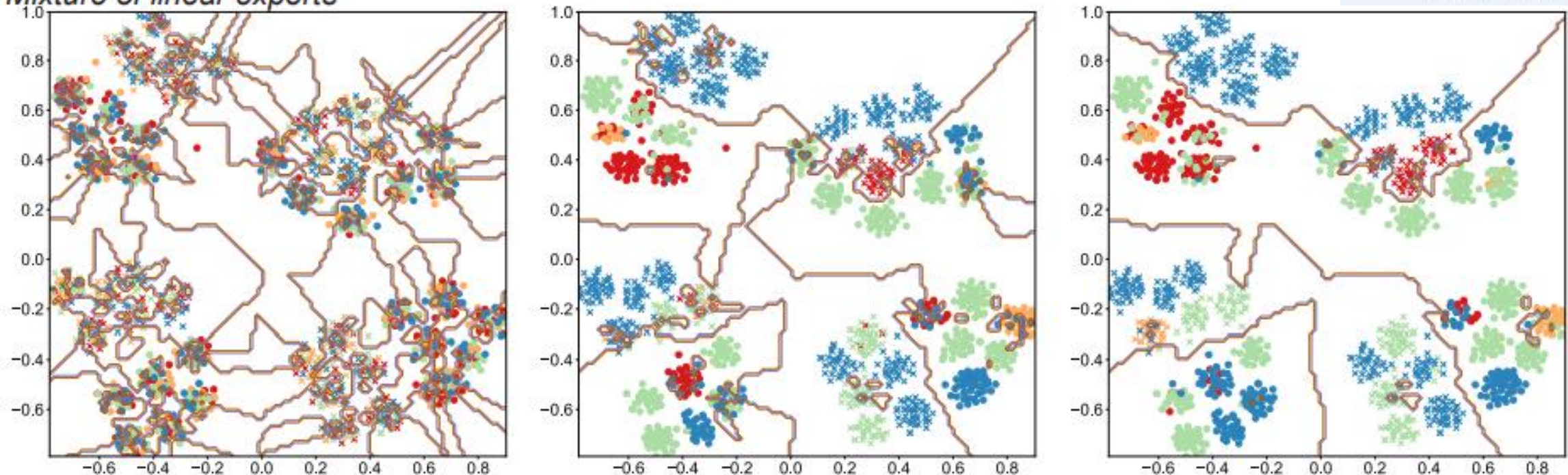
- ✓ Synthetically generated 50 dim. data
- ✓ Visualization in 2-D space
- ✓ 4 clusters
- ✓ Each cluster is linearly separable
- ✓ Ideally, a mixture of 4 linear experts sufficient for classification

Why do experts not collapse in practice?

Router dispatch:

- Blue = Expert A
- Green = Expert B
- Red = Expert C
- Orange = Expert D
- Brown lines = MoE decision boundary

Mixture of linear experts



Initialization



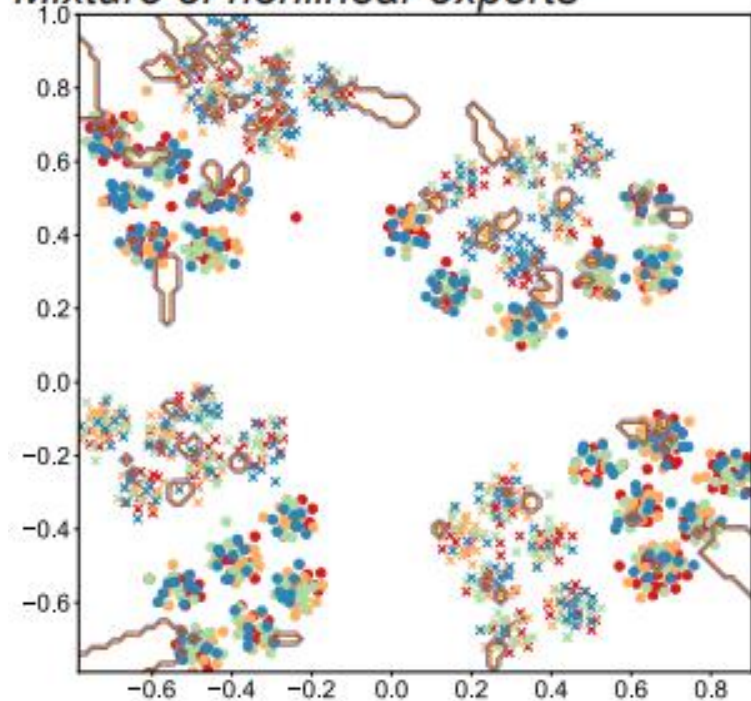
Training



Finished

Why do experts not collapse in practice?

Mixture of nonlinear experts



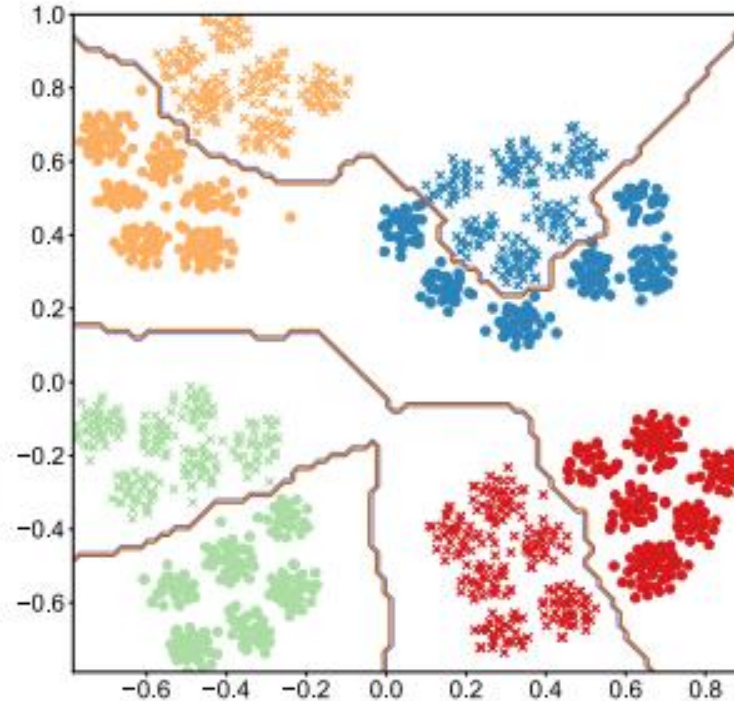
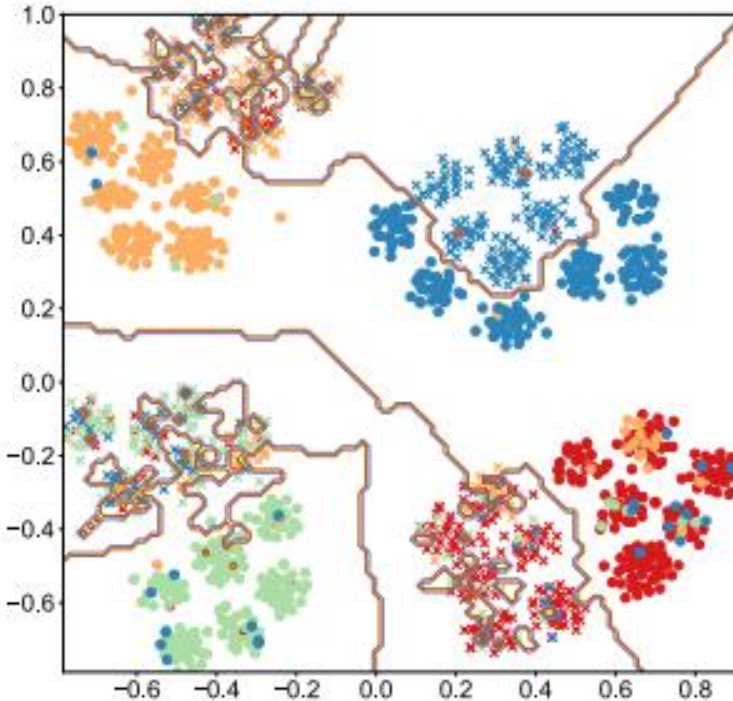
Initialization



Training



Finished

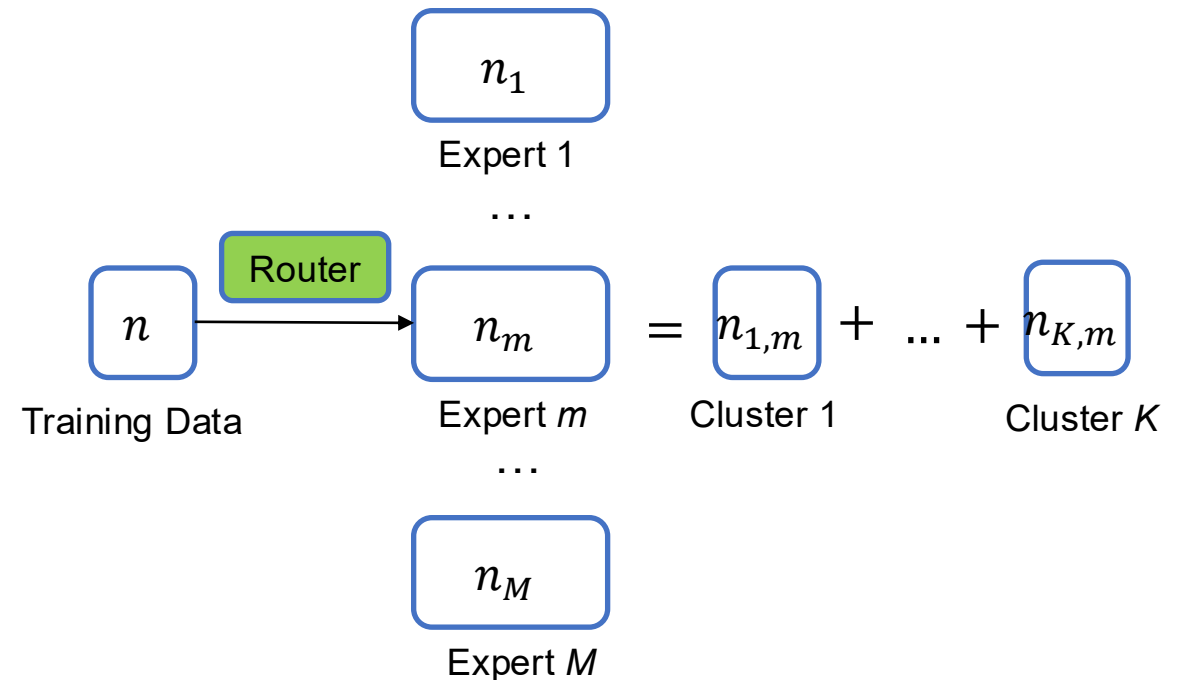


Learning dynamics of experts and router

- **Exploration stage:**
 - experts diverge; router nearly untrained
- **Router learning stage:**
 - router learns to dispatch

$$\text{entropy} = - \sum_{m=1, n_m \neq 0}^M \frac{n_m}{n} \sum_{k=1}^K \frac{n_{k,m}}{n_m} \cdot \log \left(\frac{n_{k,m}}{n_m} \right)$$

$n_{k,m}$: # of samples from Cluster k routed to Expert m
 n_m : # of samples routed to Expert $m = \sum_{k=1}^K n_{k,m}$
 n : # of total samples = $\sum_{m=1}^M n_m$

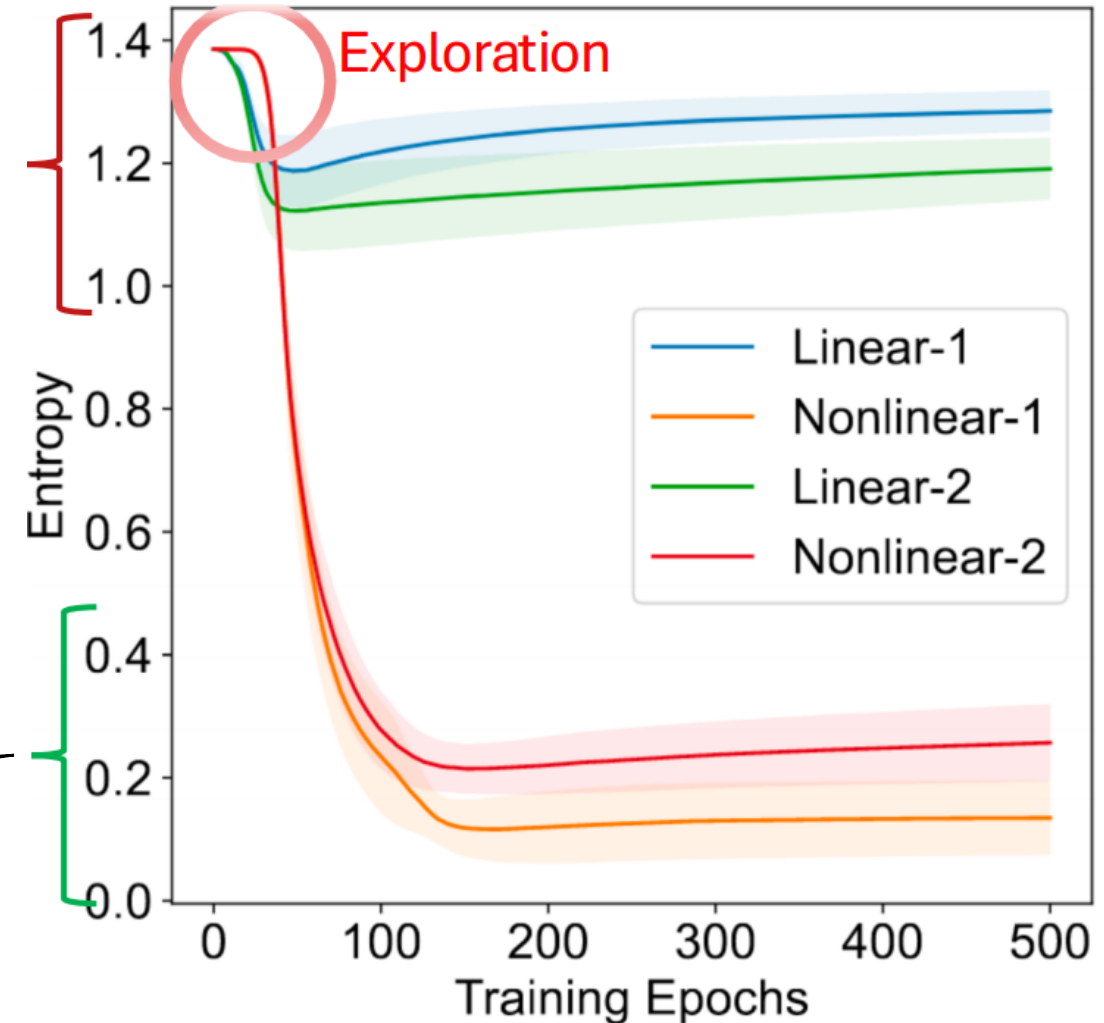


Learning dynamics of experts and router

$$\text{entropy} = - \sum_{m=1, n_m \neq 0}^M \frac{n_m}{n} \sum_{k=1}^K \frac{n_{k,m}}{n_m} \cdot \log \left(\frac{n_{k,m}}{n_m} \right)$$

Entropy is high if an input from cluster k is routed uniformly to all the experts

Entropy is low if an input from cluster k is routed to one expert



Pros and Cons of Sparse MoE Layer

Pros

- Increased model parameters
- Efficient pretraining due to conditional (sparse) computation
- Faster inference

Cons

- Unstable training
 - Router collapse – router sends all tokens to the same expert
 - May diverge
- High memory requirement - all parameters need to be loaded in GPU memory

Switch Transformer Layer

Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity

William Fedus*

LIAMFEDUS@GOOGLE.COM

Barret Zoph*

BARRETZOPH@GOOGLE.COM

Noam Shazeer

NOAM@GOOGLE.COM

Google, Mountain View, CA 94043, USA

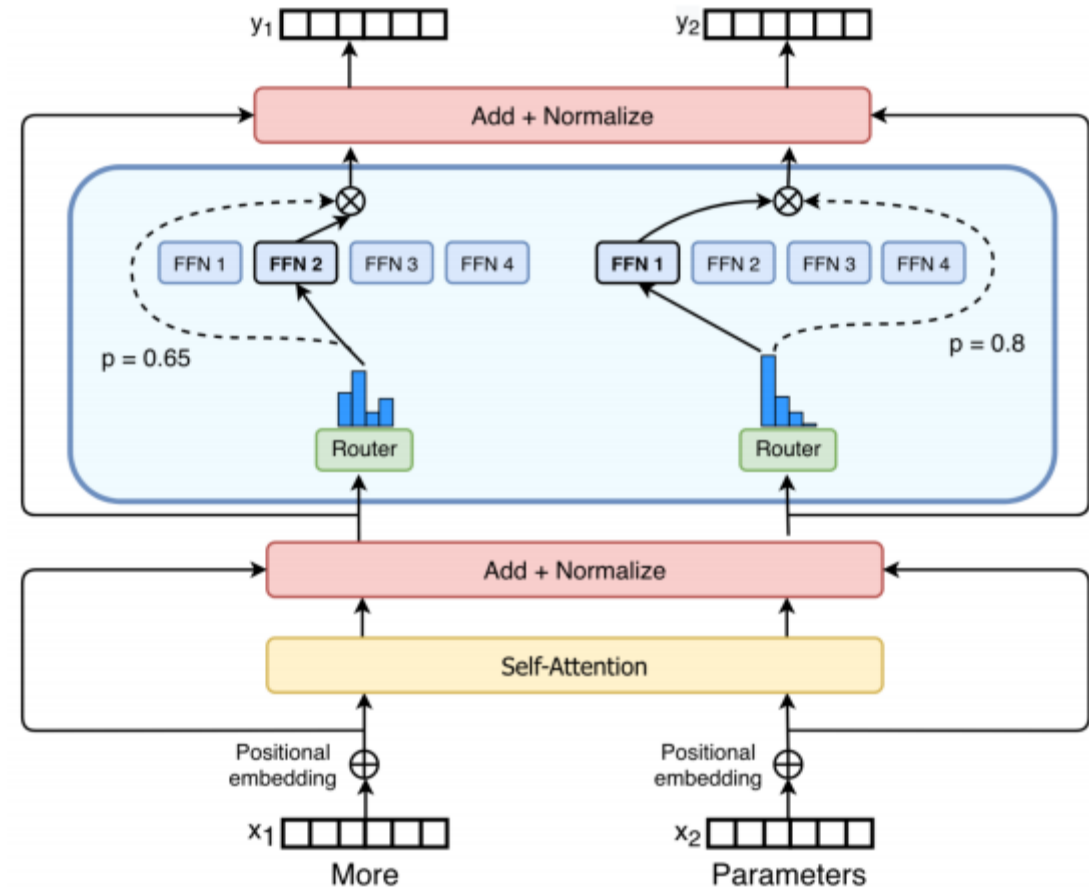
Editor: Alexander Clark

Abstract

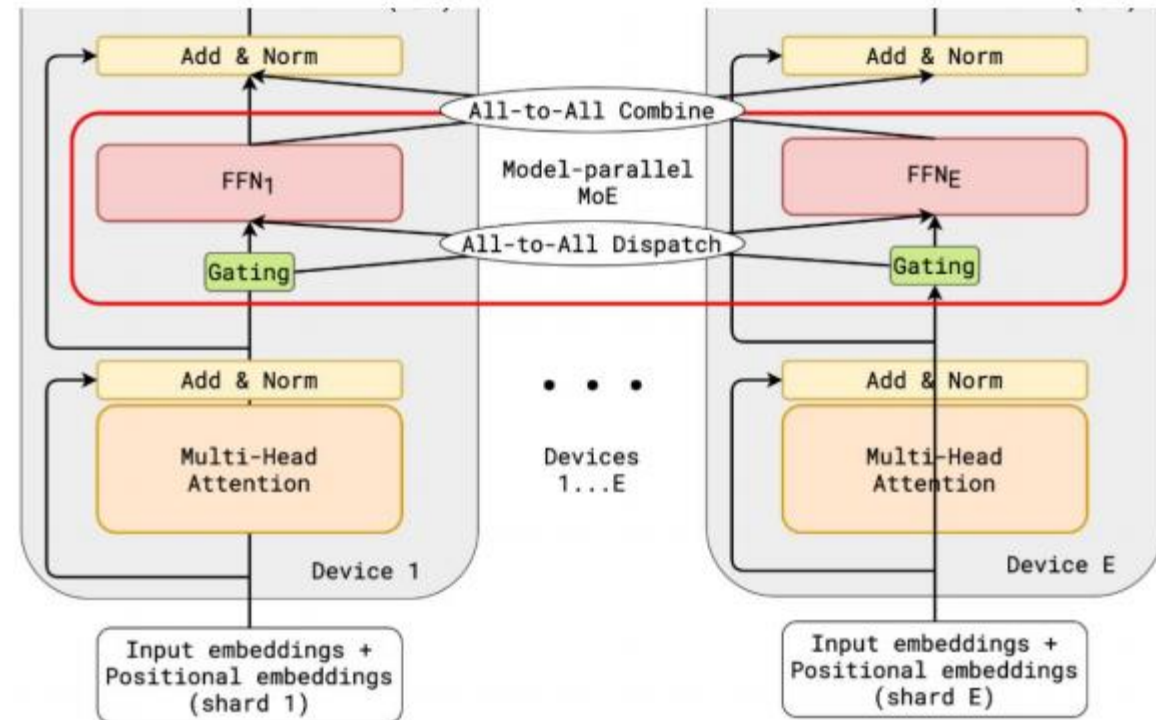
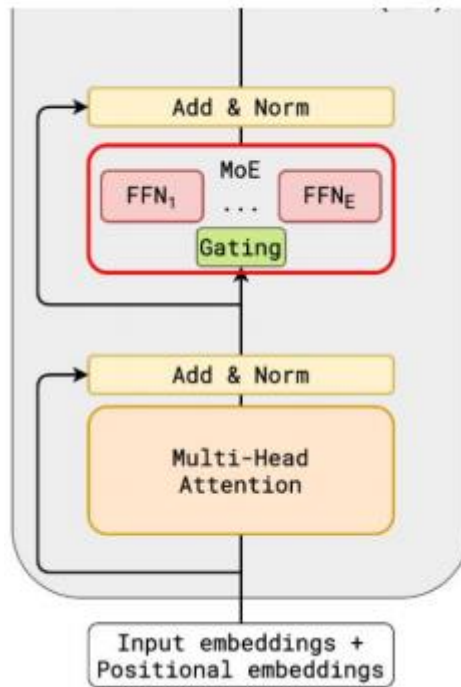
In deep learning, models typically reuse the same parameters for all inputs. Mixture of Experts (MoE) models defy this and instead select *different* parameters for each incoming example. The result is a sparsely-activated model—with an outrageous number of parameters—but a constant computational cost. However, despite several notable successes of MoE, widespread adoption has been hindered by complexity, communication costs, and training instability. We address these with the introduction of the Switch Transformer. We adapt the MoE routing algorithm and design iterative improved models with reduced

Switch Transformer Layer

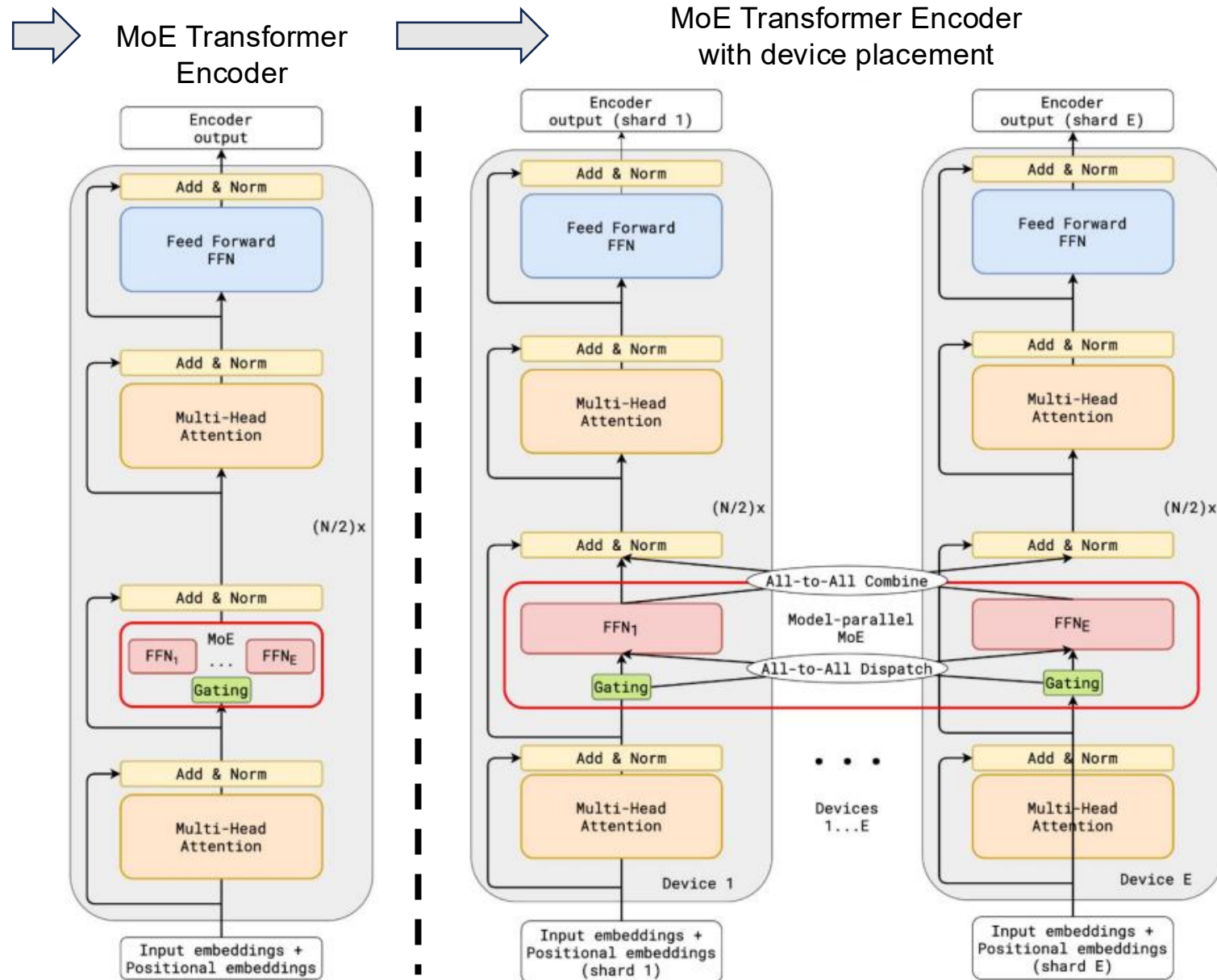
- Greedy routing to only 1 expert



Expert Parallel for Sparse MoEs

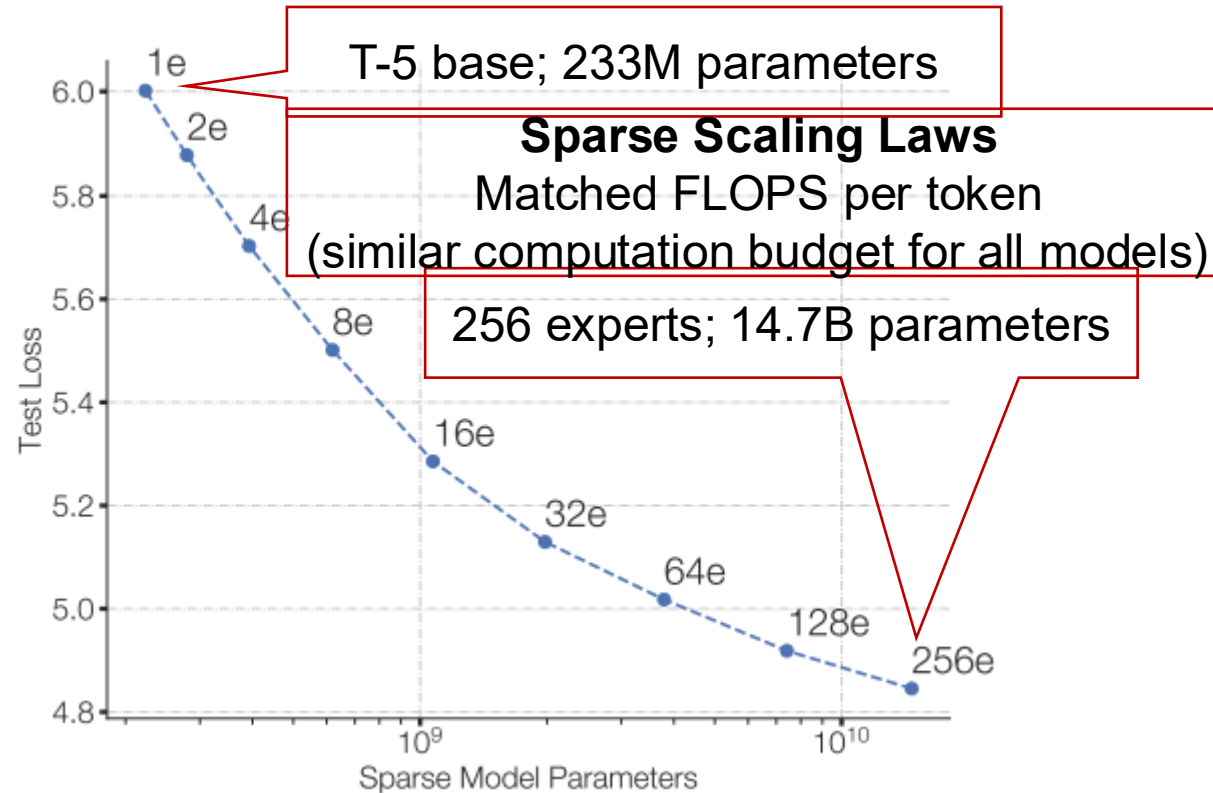


Expert Parallel for Sparse MoEs

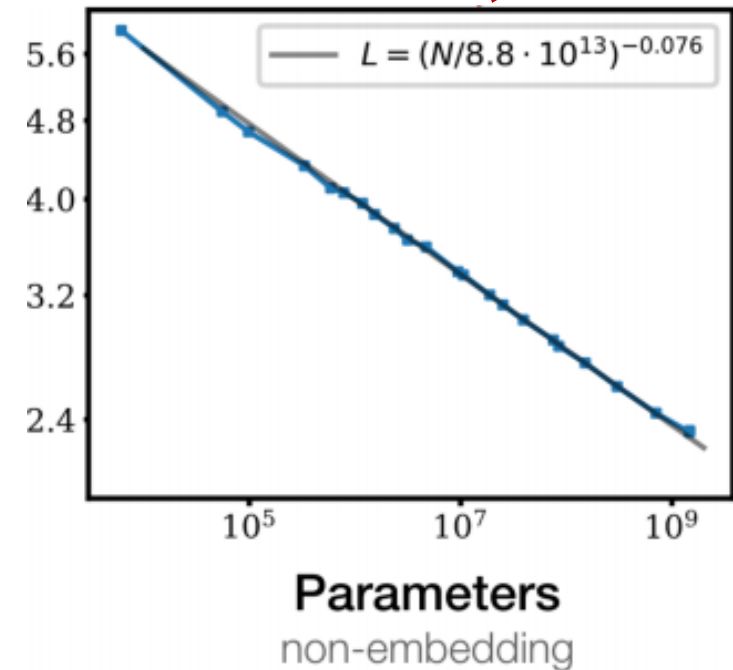


Switch Transformer Layer

- MoE-fication of T5 models



Neural Scaling Laws (unrestricted FLOPS)

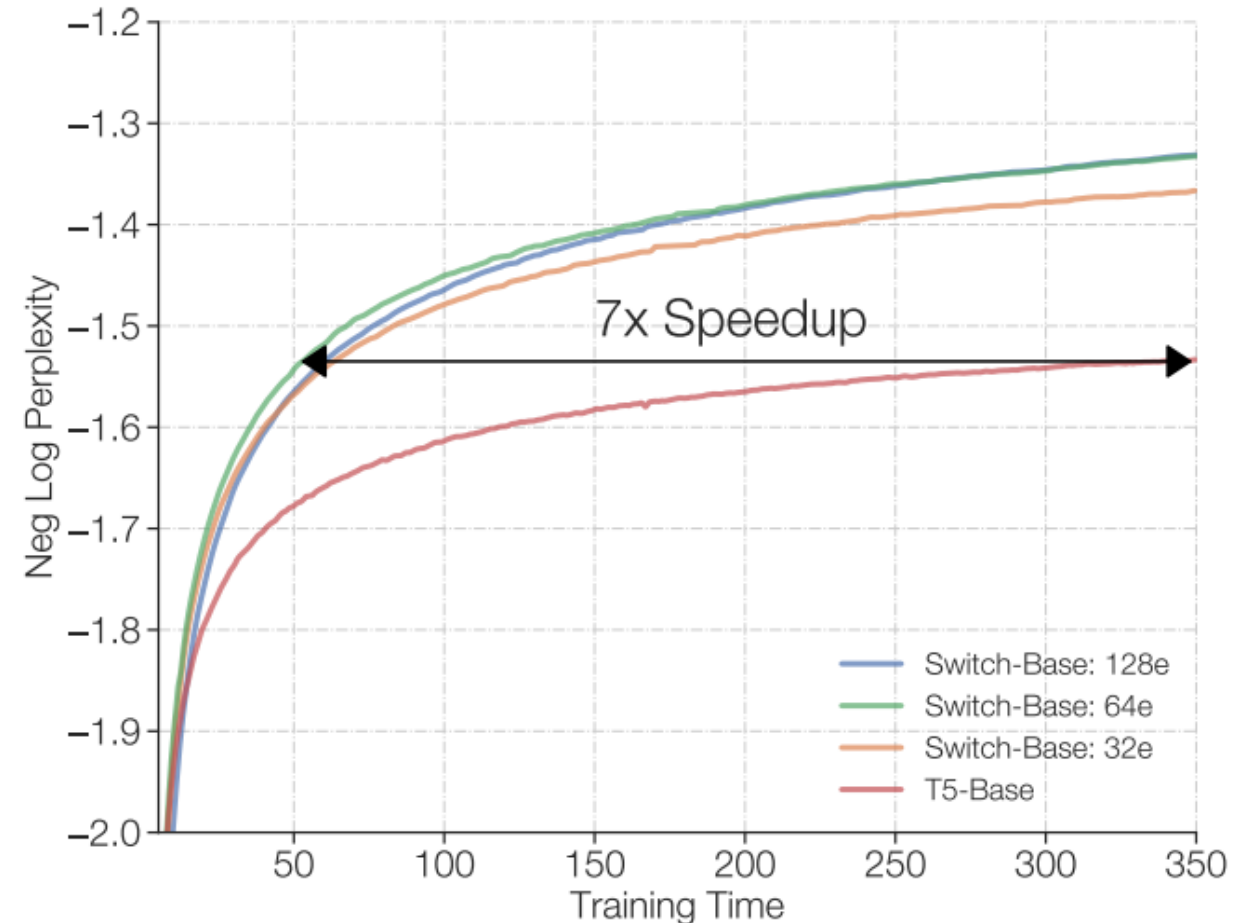


Switch Transformer Layer

- MoE-fication of T5 models

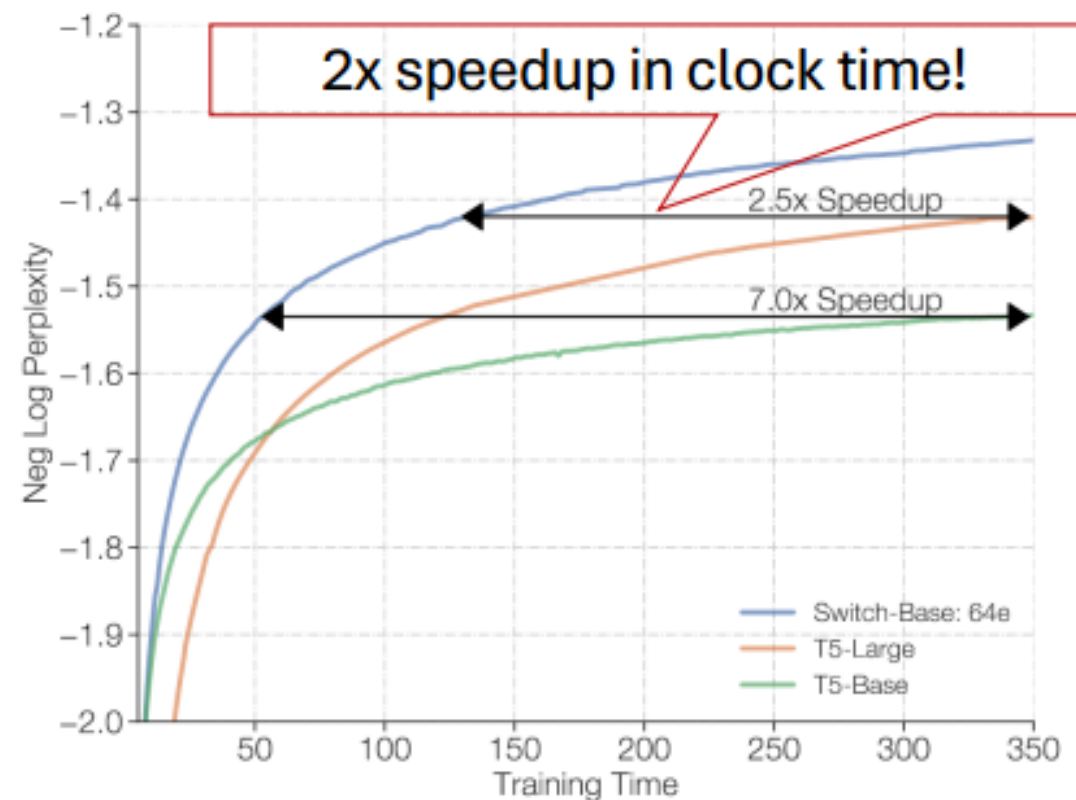
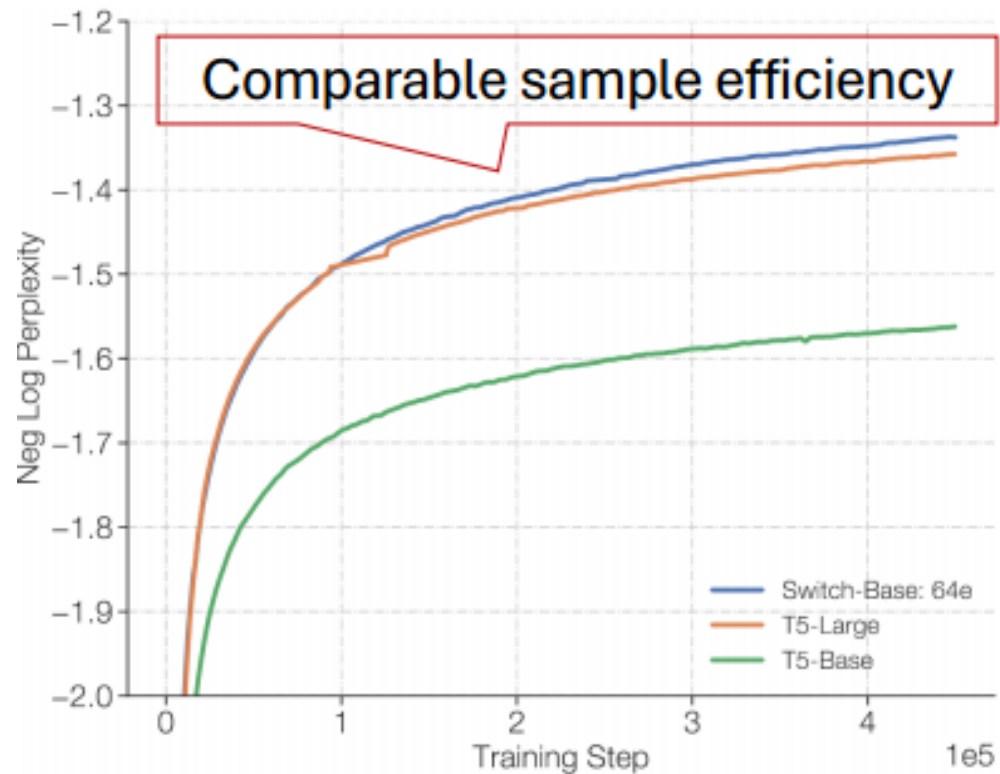
7x faster than the base model!

But what about comparison with a larger dense model?



Switch Transformer Layer

Comparison with T-5 Large (770M) using 3.5x more FLOPS per token



Switch Transformer Layer

- **Issues Addressed:**

- Complexity of MoEs
- Communication cost

Top-1 greedy routing: Challenged the belief that we need to route to at least 2 experts for meaningful learning of router

- Training Instability

Improved Training Techniques:

1. Differentiable load balancing loss (avoids router collapse)
2. Selective Precision
3. Reduced initialization scale
4. Slower learning rate warmup
5. Higher regularization of experts

Mixtral of Experts

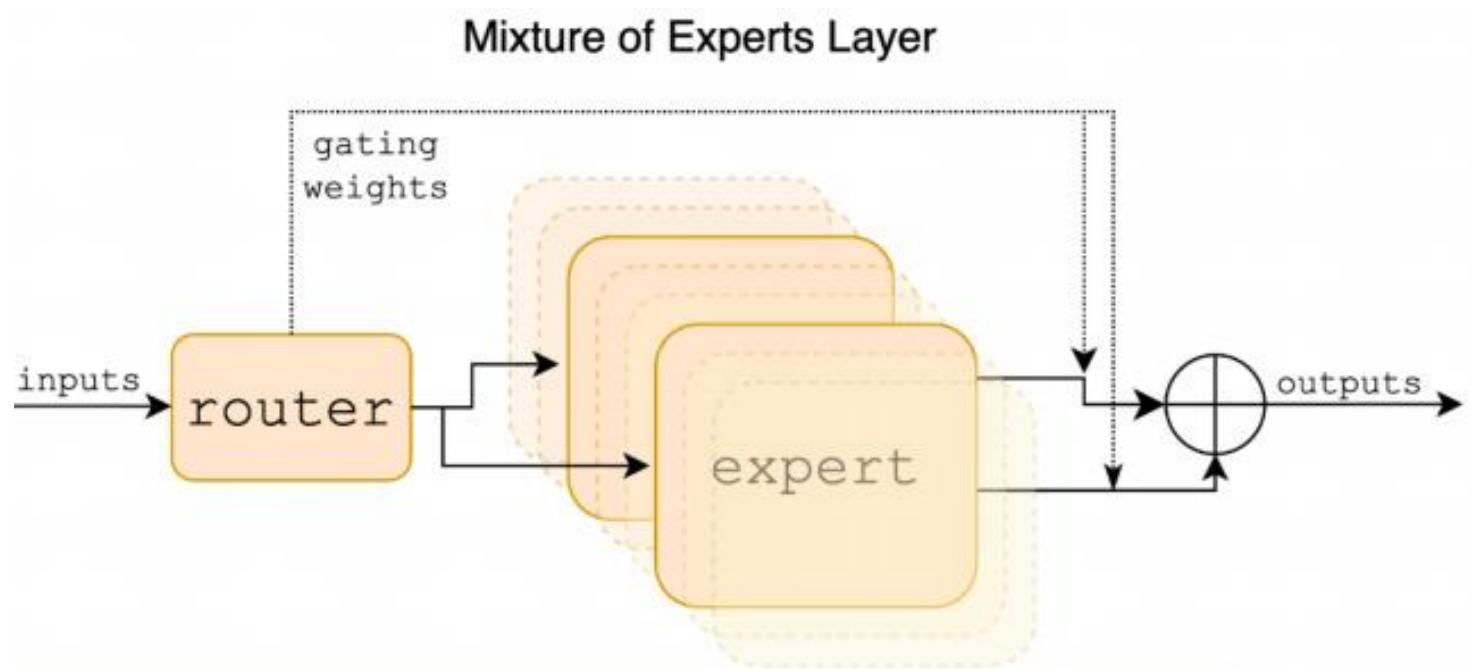


Abstract

We introduce **Mixtral 8x7B**, a Sparse Mixture of Experts (SMoE) language model. Mixtral has the same architecture as Mistral 7B, with the difference that each layer is composed of 8 feedforward blocks (i.e. experts). For every token, at each layer, a router network selects two experts to process the current state and combine their outputs. Even though each token only sees two experts, the selected experts can be different at each timestep. As a result, each token has access to **47B parameters**, but only uses **13B active parameters** during inference. Mixtral was trained with a context size of 32k tokens and it outperforms or **matches Llama 2 70B and GPT-3.5** across all evaluated benchmarks. In particular, Mixtral vastly outperforms Llama 2 70B on mathematics, code generation, and multilingual benchmarks. We also provide a model fine-

Mixture of Experts: 8x7B

Parameter	Value
dim	4096
n_layers	32
head_dim	128
hidden_dim	14336
n_heads	32
n_kv_heads	8
context_len	32768
vocab_size	32000
num_experts	8
top_k_experts	2



Replace FFN with MoE in all layers; unlike Switch Transformers

Mixture of Experts: 8x7B

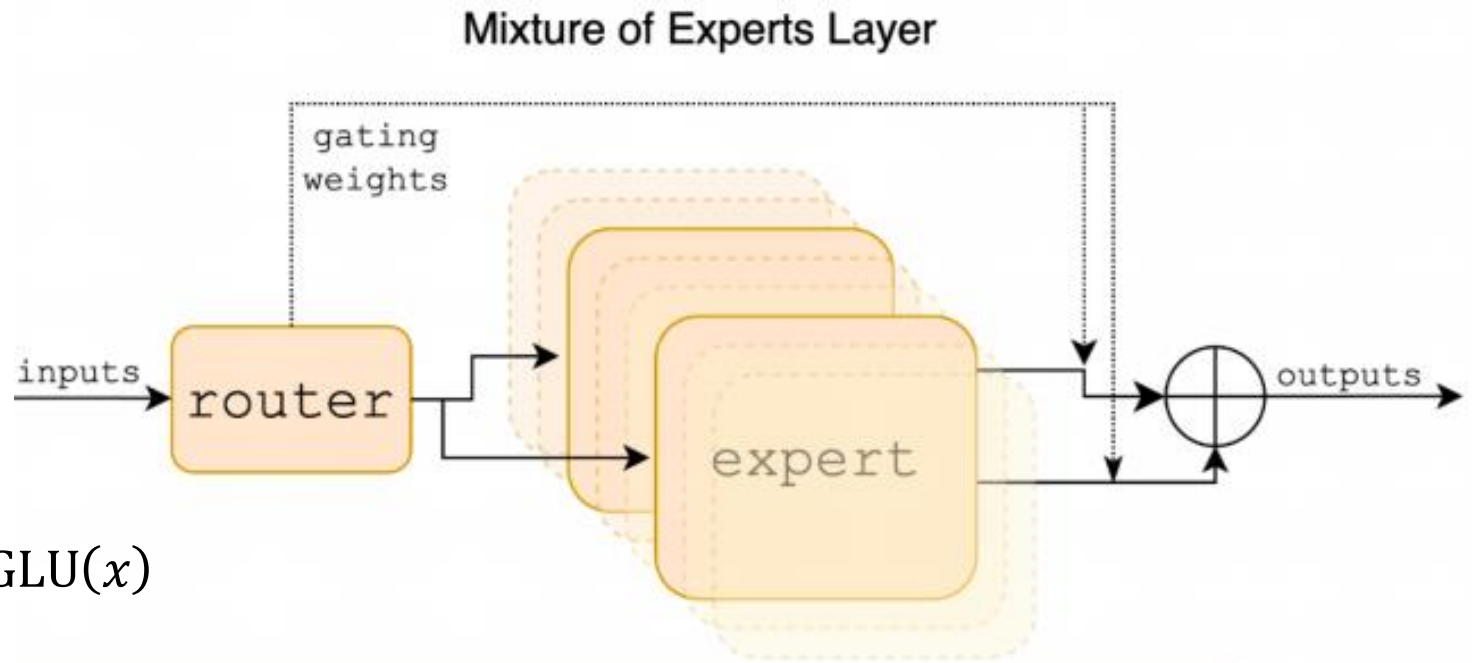
$$G(x) = \text{Softmax}(\text{TopK}(x \cdot W_g))$$

$$y = \sum_{i=0}^{n-1} G(x)_i \cdot E_i(x)$$

$$y = \sum_{i=0}^{n-1} \text{Softmax}(\text{Top2}(x \cdot W_g))_i \cdot \text{SwiGLU}(x)$$

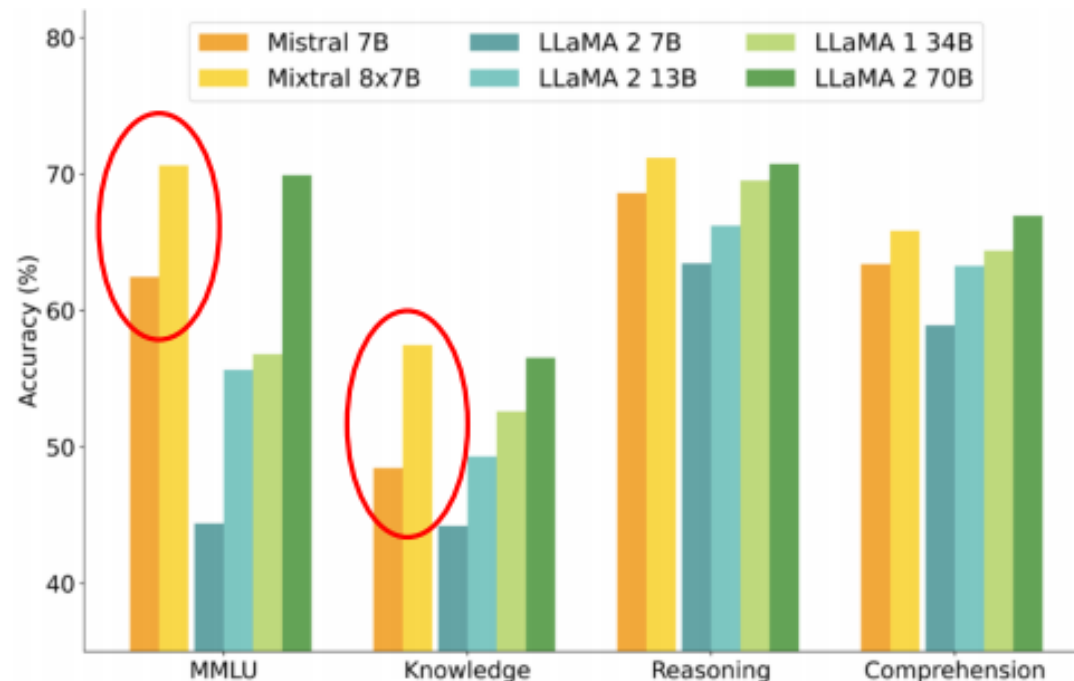
$$\text{SwiGLU}(x) = x * \text{sigmoid}(\beta * x) + (1 - \text{sigmoid}(\beta * x)) * (Wx + b)$$

Combines Swish Activation
with Gated Linear Unit (GLU)



Reasoning vs knowledge intensive tasks

- FFN layers account for knowledge
- Attention layers account for reasoning or algorithms



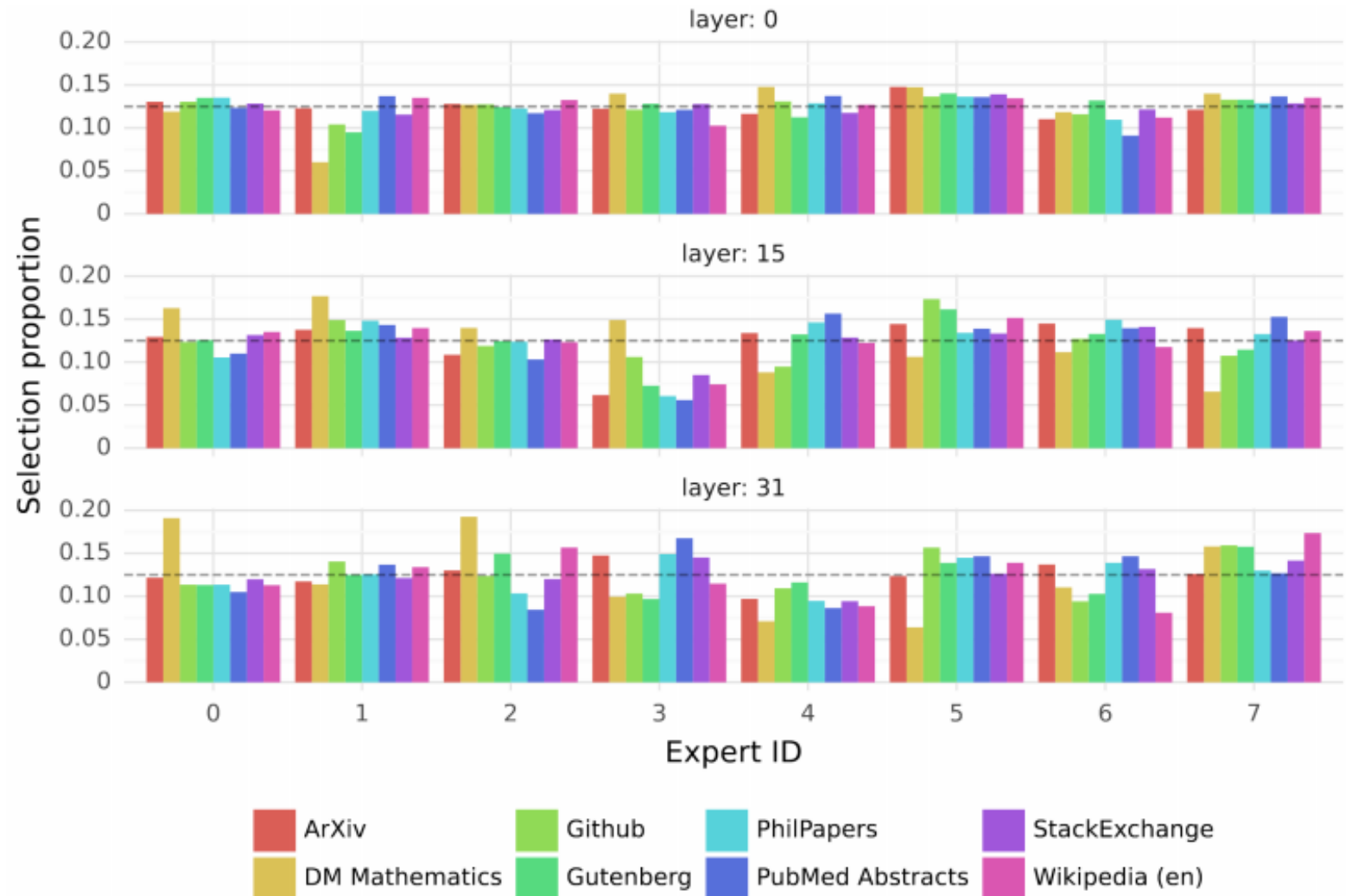
Huge gap b/w dense and corresponding sparse models on knowledge intensive tasks

Reasoning vs knowledge intensive tasks

- **Self-attention is often used as an interpretation tool**
 - Which token in the input are we attending to while generating the next token?
- **Can we use routing decisions for interpreting the model?**
 - Which tokens are routed to a particular expert?

Reasoning vs knowledge intensive tasks

- Validation split of Pile Dataset
- Proportion of tokens assigned to each expert on different domains
- Done for Layer 0, layer 15, and layer 31



Routing of Consecutive Tokens

- How many times two consecutive tokens are routed to the same expert?

- Repetitions at the first layer are close to random.
- Significantly higher at layers 15 and 31.
- The high number of repetitions shows that expert choice exhibits high temporal locality at these layers.

	Layer 0	First choice Layer 15	Layer 31
ArXiv	14.0%	27.9%	22.7%
DM Mathematics	14.1%	28.4%	19.7%
Github	14.9%	28.1%	19.7%
Gutenberg	13.9%	26.1%	26.3%
PhilPapers	13.6%	25.3%	22.1%
PubMed Abstracts	14.2%	24.6%	22.0%
StackExchange	13.6%	27.2%	23.6%
Wikipedia (en)	14.4%	23.6%	25.3%

Which experts are active for different tokens?

- Colors represent different experts
- Experts do not specialize in any domain like coding, or maths.

Layer 0

```
class MoeLayer(nn.Module):
    def __init__(self, experts: List[nn.Module],
                 super().__init__():
        assert len(experts) > 0
        self.experts = nn.ModuleList(experts)
        self.gate = gate
        self.args = moe_args

    def forward(self, inputs: torch.Tensor):
        inputs_squashed = inputs.view(-1, inputs.
        gate_logits = self.gate(inputs_squashed)
        weights, selected_experts = torch.topk(
            gate_logits, self.args.num_experts_per
        )
        weights = nn.functional.softmax(
            weights,
            dim=-1,
            dtype=torch.float,
        ).type_as(inputs)
        results = torch.zeros_like(inputs_squashed)
        for i, expert in enumerate(self.experts):
            batch_idx, nth_expert = torch.where(s
            results[batch_idx] += weights[batch_idx]
            inputs_squashed[batch_idx]
        )
        return results.view_as(inputs)
```

Question: Solve $-42*r + 27*c = -1167$ and $130*r$
Answer: 4

Question: Calculate $-841880142.544 + 411127.$
Answer: -841469015.544

Question: Let $x(g) = 9*g + 1$. Let $q(c) = 2*c +$
Answer: $54*a - 30$

A model airplane flies slower when flying into th
wind and faster with wind at its back. When launch
right angles to the wind, a cross wind, its ground
compared with flying in still air is
(A) the same (B) greater (C) less (D) either grea
or less depending on wind speed

Layer 15

```
class MoeLayer(nn.Module):
    def __init__(self, experts: List[nn.Module],
                 super().__init__():
        assert len(experts) > 0
        self.experts = nn.ModuleList(experts)
        self.gate = gate
        self.args = moe_args

    def forward(self, inputs: torch.Tensor):
        inputs_squashed = inputs.view(-1, inputs.
        gate_logits = self.gate(inputs_squashed)
        weights, selected_experts = torch.topk(
            gate_logits, self.args.num_experts_per
        )
        weights = nn.functional.softmax(
            weights,
            dim=-1,
            dtype=torch.float,
        ).type_as(inputs)
        results = torch.zeros_like(inputs_squashed)
        for i, expert in enumerate(self.experts):
            batch_idx, nth_expert = torch.where(s
            results[batch_idx] += weights[batch_idx]
            inputs_squashed[batch_idx]
        )
        return results.view_as(inputs)
```

Question: Solve $-42*r + 27*c = -1167$ and $130*r$
Answer: 4

Question: Calculate $-841880142.544 + 411127.$
Answer: -841469015.544

Question: Let $x(g) = 9*g + 1$. Let $q(c) = 2*c +$
Answer: $54*a - 30$

A model airplane flies slower when flying into th
wind and faster with wind at its back. When launch
right angles to the wind, a cross wind, its ground
compared with flying in still air is
(A) the same (B) greater (C) less (D) either grea
or less depending on wind speed

Layer 31

```
class MoeLayer(nn.Module):
    def __init__(self, experts: List[nn.Module],
                 super().__init__():
        assert len(experts) > 0
        self.experts = nn.ModuleList(experts)
        self.gate = gate
        self.args = moe_args

    def forward(self, inputs: torch.Tensor):
        inputs_squashed = inputs.view(-1, inputs.
        gate_logits = self.gate(inputs_squashed)
        weights, selected_experts = torch.topk(
            gate_logits, self.args.num_experts_per
        )
        weights = nn.functional.softmax(
            weights,
            dim=-1,
            dtype=torch.float,
        ).type_as(inputs)
        results = torch.zeros_like(inputs_squashed)
        for i, expert in enumerate(self.experts):
            batch_idx, nth_expert = torch.where(s
            results[batch_idx] += weights[batch_idx]
            inputs_squashed[batch_idx]
        )
        return results.view_as(inputs)
```

Question: Solve $-42*r + 27*c = -1167$ and $130*r$
Answer: 4

Question: Calculate $-841880142.544 + 411127.$
Answer: -841469015.544

Question: Let $x(g) = 9*g + 1$. Let $q(c) = 2*c +$
Answer: $54*a - 30$

A model airplane flies slower when flying into th
wind and faster with wind at its back. When launch
right angles to the wind, a cross wind, its ground
compared with flying in still air is
(A) the same (B) greater (C) less (D) either grea
or less depending on wind speed

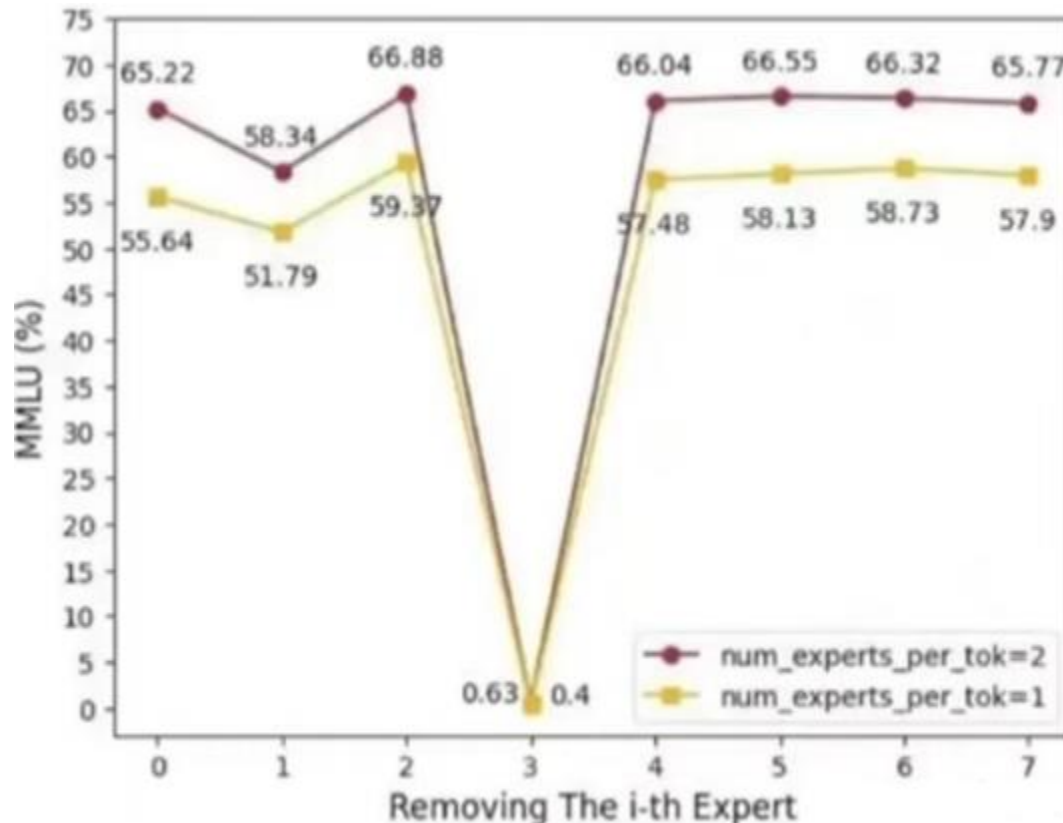
Coding
question

Arithmetic
question

MCQ
question

Interpreting experts

- There is one expert in one of the layers that's particularly crucial.



References

- Chen et al., “Towards Understanding Mixture of Experts in Deep Learning”. NeurIPS 2022
- Shazeer et al., “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-Of-Experts Layer”. ICLR 2017
- Fedus et al., “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity”. JMLR 23 (2022) 1-40
- Jiang et al., “Mixtral of Experts”. arXiv:2401.04088